

자기장센서를 활용한 직선거리
추출시스템

신한대학교
전자공학과
20191375 이성녕

목차

I. 서론	8
1. 개발 배경	8
2. 개발 목적	8
3. 개발 범위	8
4. 개발 방법	9
5. 논문 구성	9
II. 이론	10
1. I2C(Inter-Integrated Circuit)	10
가. 시작 조건, 종료 조건	10
나. 주소 패킷 포맷	11
다. 데이터 패킷 포맷	11
라. 일반적인 데이터 송수신 형태	12
2. UART(Universal Asynchronous Receiver Transmitter)	13
가. 통신속도(Baud Rate)	14
나. 프레임 포맷	14
3. NMEA(The National Marine Electronics Association) 프로토콜	15
가. GPGGA(Global Positioning System Fix Data)	16
나. GPRMC(Recommended Minimum Data)	16
4. 지구 자기장을 이용한 방위각(Heading)	17
가. 자북(Magnetic North)과 진북(True North)	17
나. 지구 자기장을 이용한 방위각 계산	18
5. 지구 위에서 두 좌표 사이의 거리와 방위각(Bearing)	19
가. 하버사인(Haversine)의 공식	19
나. 빈센티(Vincenty)의 공식	19
III. 시스템 구현	21
1. 하드웨어 구현	21
가. 시스템 블록도	21
나. 포트할당	22
다. 보조 회로도	23
2. 소프트웨어 구현	24
가. 4x3 Key Matrix	24
나. SSD1309 OLED Display	26
다. HMC5883L	29

라. NEO-6M	31
마. 통합 알고리즘	33
IV. 실험 및 결과	35
1. 기준좌표(신한대학교)	35
2. 실험	36
가. 기준좌표에서 춘천시청까지의 실험 결과	37
나. 기준좌표에서 대전광역시청까지의 실험 결과	39
다. 기준좌표에서 대구광역시청까지의 실험 결과	40
라. 도쿄 대학교에서 괴팅겐 대학교까지의 실험 결과	41
마. 하버드 대학교에서 케임브리지 대학교까지의 실험 결과	42
바. 모스크바 국립대학교에서 소르본 대학교까지의 실험 결과	42
3. 실험결과 정리 및 오차율 계산	43
V. 결론	47
참고문헌	48

그림 목차

그림 1. I2C 통신에서 마스터와 슬레이브 그리고 풀업저항	10
그림 2. I2C 통신의 시작과 종료 파형	11
그림 3. I2C 통신의 주소 패킷 포맷	11
그림 4. I2C 통신의 데이터 패킷 포맷	12
그림 5. I2C 쓰기 예시	12
그림 6. I2C 읽기 예시	13
그림 7. I2C 통신의 일반적인 데이터 송수신 형태	13
그림 8. UART 통신에서 디바이스간 포트 연결	14
그림 9. ATmega128의 UART 통신속도 관련 공식	14
그림 10. UART 통신에서 가능한 프레임 포맷	15
그림 11. NMEA 프로토콜 프레임	15
그림 12. GPGGA 문장의 형식	16
그림 13. GPRMC 문장의 형식	17
그림 14. 자북과 진북 비교	18
그림 15. 직교 좌표계에서 지구 자기장의 표현	18
그림 16. 하버사인의 거리 공식	19
그림 17. 하버사인의 방위각 공식	19
그림 18. WGS 84의 규격	20
그림 19. 빈센티의 공식	20
그림 20. 전체 시스템 블럭도	22
그림 21. 키 매트릭스와 LED에 대한 보조 회로도	24
그림 22. 키 매트릭스 응용 순서도	26
그림 23. 세계지도 원본	27
그림 24. 한반도 지도 원본	27
그림 25. SSD1309 응용 순서도	28
그림 26. 세계지도 출력 테스트	28
그림 27. 한반도 출력 테스트	28
그림 28. HMC5883L 응용 순서도	30
그림 29. 나침반과 자기장 센서의 측정값 비교	31
그림 30. NEO-6M 응용 순서도	32
그림 31. NEO-6M 동작 테스트	32
그림 32. 통합 알고리즘 순서도	34
그림 33. 공식 변환 순서도	34
그림 34. 세계지도 모드 출력	35

그림 35. 한반도 모드 출력	35
그림 36. 세계지도 모드와 한반도 모드 출력 비교	36
그림 37. 기준좌표에서 춘천시청까지의 구글지도 결과	38
그림 38. 기준좌표에서 춘천시청까지의 하버사인 모드 결과	38
그림 39. 기준좌표에서 춘천시청까지의 빈센티 모드 결과	38
그림 40. 기준좌표에서 춘천시청까지의 Vincenty's Inverse 결과	38
그림 41. 기준좌표에서 대전광역시청까지의 구글지도 결과	39
그림 42. 기준좌표에서 대전광역시청까지의 하버사인 모드 결과	39
그림 43. 기준좌표에서 대전광역시청까지의 빈센티 모드 결과	39
그림 44. 기준좌표에서 대전광역시청까지의 Vincenty's Inverse 결과	40
그림 45. 기준좌표에서 대구광역시청까지의 구글지도 결과	40
그림 46. 기준좌표에서 대구광역시청까지의 하버사인 모드 결과	41
그림 47. 기준좌표에서 대구광역시청까지의 빈센티 모드 결과	41
그림 48. 기준좌표에서 대구광역시청까지의 Vincenty's Inverse 결과	41
그림 49. 도쿄 대학교에서 괴팅겐 대학교까지의 하버사인 모드 결과	41
그림 50. 도쿄 대학교에서 괴팅겐 대학교까지의 빈센티 모드 결과	41
그림 51. 도쿄 대학교에서 괴팅겐 대학교까지의 Vincenty's Inverse 결과	41
그림 52. 하버드 대학교에서 케임브리지 대학교까지의 하버사인 모드 결과	42
그림 53. 하버드 대학교에서 케임브리지 대학교까지의 빈센티 모드 결과	42
그림 54. 하버드 대학교에서 케임브리지 대학교까지의 Vincenty's Inverse 결과 ..	42
그림 55. 모스크바 국립대학교에서 소르본 대학교까지의 하버사인 모드 결과	43
그림 56. 모스크바 국립대학교에서 소르본 대학교까지의 빈센티 모드 결과	43
그림 57. 모스크바 국립대학교에서 소르본 대학교까지의 Vincenty's Inverse 결과	43

표 목차

표 1. 사용자 인터페이스 포트할당	22
표 2. 자기장 센서 및 GPS 모듈 포트할당	23
표 3. OLED 디스플레이 위치에 따른 GPS 좌표	29
표 4. GPS 좌표에 따른 OLED 디스플레이의 수평선 위치 공식	29
표 5. GPS 좌표에 따른 OLED 디스플레이의 수직선 위치 공식	29
표 6. 기준좌표(신한대학교) 측정	36
표 7. 기준좌표 및 국내 주요 시청 좌표	37
표 8. 해외 주요 대학 좌표	37
표 9. 거리 비교	44
표 10. 거리 오차 및 오차율	44
표 11. 방위각 비교	45
표 12. 방위각 오차 및 오차율	45
표 13. 국내에서의 하버사인 모드 오차율 평균값	46
표 14. 국내에서의 빈센티 모드 오차율 평균값	46
표 15. 해외에서의 하버사인 모드 오차율 평균값	46
표 16. 해외에서의 빈센티 모드 오차율 평균값	46

요약문

본 논문은 지구 위에서 두 GPS 좌표 사이의 거리와 방위각을 추출할 수 있는 시스템의 설계 및 구성 요소에 관하여 기술한다. 현재 두 GPS 좌표 사이의 거리와 방위각을 추출할 수 있는 공식은 현재 하버사인의 공식과 빈센티의 공식 2개가 알려져 있다. 해당 시스템은 사용자가 처한 환경에 따라 혹은 사용하는 용도에 두 공식 중 하나를 직접 선택할 수 있게 설계되었다. 따라서 본 논문은 사용자가 계산에 사용할 공식을 선택할 수 있게 설계함으로써 두 공식이 갖고있는 장점만을 취할 수 있는, 범용적인 거리 및 방위각 계산기 시스템을 논의할 것이다.

I. 서론

1. 개발 배경

기술이 발전함에 따라 소비자는 하나의 시스템에 대하여 정확성과 속도 간의 균형, 유연한 사용자 경험 제공, 리소스 절약 등의 다양한 산업적 특성을 요구하고 있다. 현대 사회는 글로벌 사회로 GPS를 응용한 위치기반 시스템이 군사, 물류, 항공, 운송, GIS, IoT 등 다양한 산업에서 사용되고 있다. 그리고 위치기반 시스템은 하나의 좌표만으로는 의미가 없으니 반드시 GPS 수신기의 위치와 사용자가 입력한 다른 위치 사이의 거리와 방위각을 계산하는 공식이 탑재되어있다. 현재 두 좌표 사이의 거리와 방위각을 계산하는 공식은 하버사인의 공식과 빈센티의 공식이 알려져있다.

2. 개발 목적

하버사인의 공식은 계산량이 적지만 일부 오차를 허용해야한다. 반면에 빈센티의 공식은 계산량이 많지만 오차가 거의 없다. 정확도 측면에서 빈센티의 공식만 사용하여 시스템을 만들어도 괜찮지만, 하버사인의 공식도 대부분의 상황에서 상당히 높은 정확도를 갖고있으며 계산량이 훨씬 적기 때문에 사용자가 거리와 방위각 추출 목적에 따라 두 공식을 적절히 선택하여 사용할 수 있다면, 그 시스템이 각각의 공식의 장점만을 취하여 훨씬 범용적으로 사용될 수 있을 것이라 생각하여 개발하게 되었다.

3. 개발 범위

이 시스템은 입력 대기 상태에서는 GPS 시계와 같이 동작하다가 좌표가 입력되면 두 GPS 좌표 사이의 거리와 방위각을 계산하기 시작한다. 또한 키 매트릭스를 이용한 커맨드 입력 기능을 이용하여 3가지 부가 기능을 추가하였다. 위의 기능들을 정리하면 다음과 같다.

첫째, 입력 대기 상태에서는 GPS 시계로서 동작한다.

둘째, 추적 상태에서 Heading과 Bearing을 맞춘 후 Distance만큼 앞으로 전진하면 목적지에 도달할 수 있다.

셋째, 자기편각을 사용자가 직접 입력할 수 있다.

넷째, 사용자가 세계지도/한반도 모드를 직접 선택할 수 있다.

다섯째, 사용자가 하버사인/빈센티 모드를 직접 선택할 수 있다.

GPS 모듈을 통해 UTC 날짜 및 시간 그리고 사용자의 현재 위치를 수신한다. 수신된 정보는 가공 혹은 그대로 출력하는 데에 사용한다. Heading은 사용자의 현재 진행 방향으로 자기장 센서가 측정한 지구의 자기장과 자기편각을 참고하여 계산한다. 또한 사용자가 두 GPS 좌표에 대해 이해하기 쉽도록 OLED 디스플레이를 추가하여 지도 그래픽위에서 두 좌표 사이의 거리를 대략적으로 확인할 수 있게 하였다.

4. 개발 방법

펌웨어 개발은 Windows OS 환경에서 C로 개발하며, 통합 개발 환경은 Microchip Studio로 한다. 마이크로 컨트롤러는 Microchip Technology의 ATmega128을 선택한다. 부품은 크게 사용자 인터페이스용 부품, 자기장 센서, GPS 모듈로 분류된다. 사용자 인터페이스용 부품으로는 4x3 Key Matrix, 20x04 Character LCD, SSD1309 OLED Display, 서로 색이 다른 LED 2개가 있다. 자기장 센서로는 HMC5883L, GPS 모듈로는 NEO-6M을 사용한다.

5. 논문 구성

본 논문의 나머지 부분은 다음과 같이 구성된다. II 장에서는 본 시스템에 사용된 기술 그리고 거리 및 방위각 추출과 관련된 이론에 대하여 살펴볼 것이다. III 장에서는 시스템 구성요소에 해당하는 각각의 부품들에 II 장의 기술이 어떻게 적용되었는지, 각각의 부품들을 이용하여 이론의 계산을 어떻게 수행할 수 있는지 살펴볼 것이다. 또한 각각의 부품들이 함께 동작할 수 있는 통합 알고리즘을 마지막에 살펴볼 것이다. IV 장에서는 해당 시스템을 이용해 측정한 수치 데이터를 이용해 오차율을 계산하여 해당 시스템의 정확도를 살펴볼 것이다. V 장에서는 해당 시스템을 통해 이론 결과와 문제점에 대하여 살펴본 후 논문을 마친다.

II. 이론

1. I2C(Inter-Integrated Circuit)

I2C 통신은 대표적인 디지털 통신 방법 중 하나이면서 동시에 직렬 통신 방법이다. 본 논문에서 다루는 I2C 외 디지털 직렬 통신 방법인 UART와의 차이는 그림 1과 같이 여러 개의 슬레이브가 하나의 마스터의 SCL, SDA 버스선을 공유한다는 점이다. 이러한 점은 각각의 슬레이브가 서로 다른 7비트 주소를 가질 때 가능해지며, 본격적인 통신을 시작하기 전 마스터가 통신하고자 하는 슬레이브의 주소를 미리 송신하여 통신하고자 하는 슬레이브만을 활성화한다. 또한 I2C 통신은 그림 1과 같이 풀업저항을 이용하여 디지털 신호를 생성하는데 ATmega128의 경우 내부적인 풀업저항이 존재하기 때문에 사용하는 슬레이브 디바이스의 데이터시트를 참고하여 ATmega128의 내부 풀업저항과 값이 같으면 별도의 풀업저항을 달아줄 필요 없다. I2C 통신은 2개의 버스선을 이용하여 통신 시작 및 종료, 주소 송수신, 데이터 송수신을 해야하며 각각 포맷이 존재한다. 이와같은 포맷은 다음과 같다.

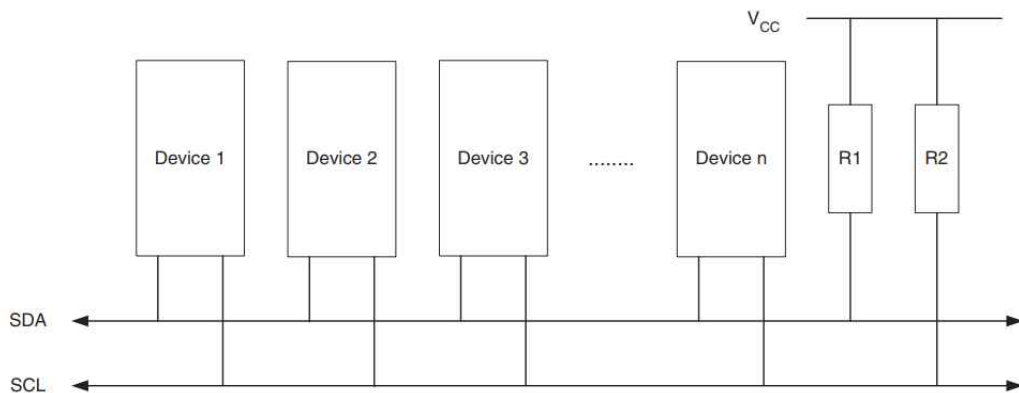


그림 1. I2C 통신에서 마스터와 슬레이브 그리고 풀업저항

가. 시작 조건, 종료 조건

I2C는 SCL의 High, Low 그리고 SDA의 상승 엣지, 하강 엣지로 통신의 시작과 종료를 결정한다. 그림 2와 같이 SCL이 High일 때 SDA이 하강 엣지이면 통신을 시작하고, SCL이 High일 때 SCL이 상승엣지이면 통신을 종료한다.

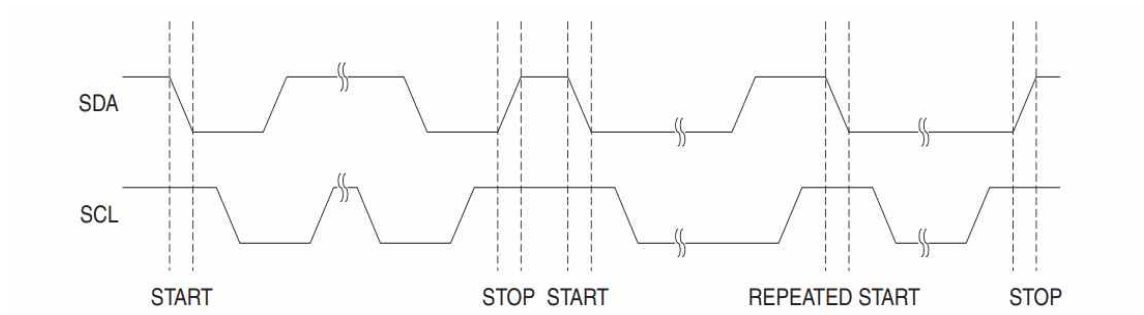


그림 2. I2C 통신의 시작과 종료 파형

나. 주소 패킷 포맷

슬레이브의 주소는 앞서 언급했던바와 같이 7비트로 구성되어있는데 I2C 통신은 한번 통신할 때 8비트의 데이터를 송수신한다. 따라서 마스터에서 통신하고자하는 슬레이브의 주소를 보낼 때 남은 1비트에 마스터 측에서 슬레이브의 레지스터 값을 읽을건지, 슬레이브의 레지스터에 값을 쓸건지 결정하는 비트를 추가하여 8비트를 만들어 송신한다. 그리고 그림 3과 같이 마스터가 [주소+R/W]를 송신하면 주소에 해당하는 슬레이브는 마스터에게 정상적으로 데이터를 수신했다는 신호인 ACK를 송신한다.

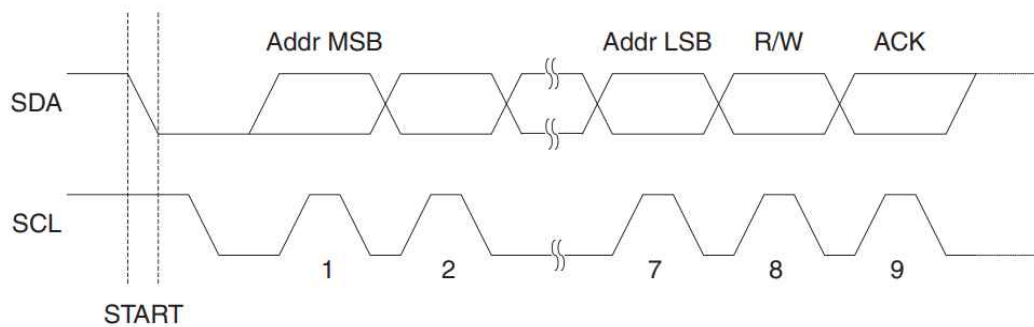


그림 3. I2C 통신의 주소 패킷 포맷

다. 데이터 패킷 포맷

위와 같이 마스터가 [주소+W]를 송신했을 때 마스터는 슬레이브에게 2가지 행동을 할 수 있다. 각각 슬레이브의 현재 포인터가 특정 주소의 레지스터를 가리키고 있게 하는 것, 만약 현재 포인터가 특정 주소의 레지스터를 가리키고 있을 때 그 레지스터에 특정 값을 저장하는 것이다.

만약 마스터가 [주소+R]을 송신했을 때 마스터는 현재 슬레이브의 포인터가 가리키고 있는 레지스터에 저장된 값을 읽어온다. 만약 마스터가 정상적으로 데이터를 수신했을 때 슬레이브에게 ACK(Acknowledge Character)를 송신하며 타이밍 다이어그램으로 표현하면 그림 4와 같다. 만약 마스터가 정상적으로 데이터를 수신하지 못했을 때 슬레이브에게 NACK(Negative Acknowledge)를 송신한다. 그러나 ACK와 NACK는 단순히 제대로 수신했다는 신호를 보내는 것 외에도 용도가 하나 더 있다. 그것은 슬레이브의 포인터가 현재 레지스터에서 다음 레지스터로 이동시키는 것과 현재 레지스터에 머물게하는 것이다. 마스터가 ACK를 송신할 경우 슬레이브의 포인터는 현재 레지스터에서 다음 레지스터로 이동하며, NACK를 송신할 경우 슬레이브의 포인터는 현재 레지스터에 머물게 된다.

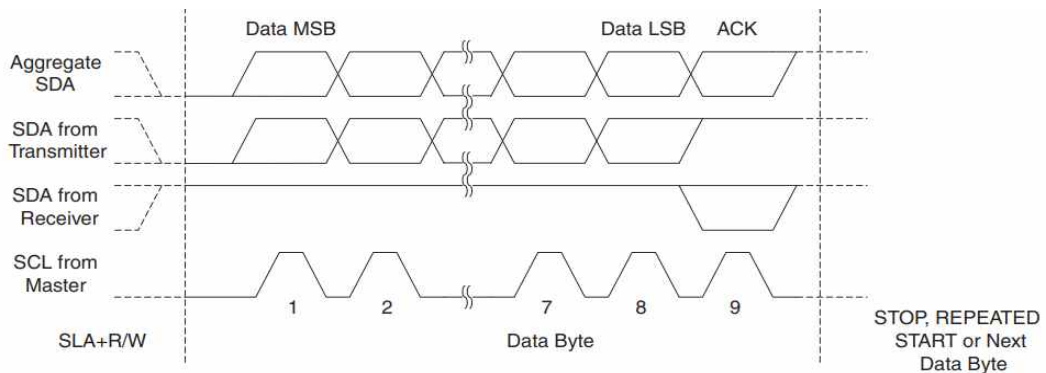


그림 4. I2C 통신의 데이터 패킷 포맷

라. 일반적인 데이터 송수신 형태

앞의 내용을 참고하여 슬레이브의 특정 레지스터에 특정 값을 저장하고자 할 때 마스터는 그림 5와 같이 통신하면 된다.



그림 5. I2C 쓰기 예시

또한 슬레이브의 특정 레지스터 2개를 연속으로 읽고자 할 때 마스터는 그림 6과 같이 통신하면 된다.

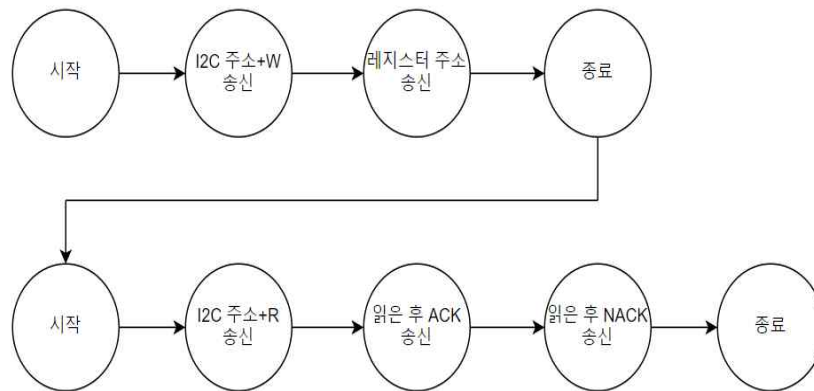


그림 6. I2C 읽기 예시

그림 5와 그림 6의 내용을 타이밍 다이어그램으로 표현하면 그림 7과 같이 표현될 수 있으며 그림 7이 I2C 통신의 일반적인 데이터 송수신 형태이다.

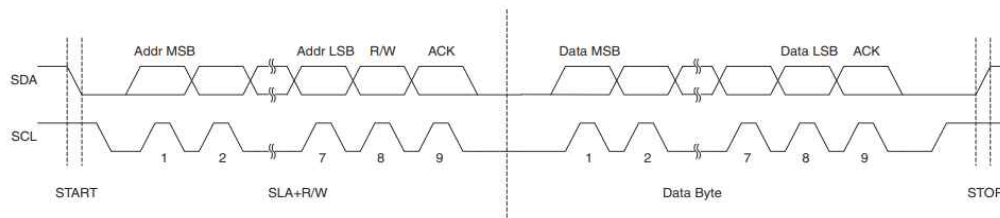


그림 7. I2C 통신의 일반적인 데이터 송수신 형태

2. UART(Universal Asynchronous Receiver Transmitter)

UART는 I2C와 같이 대표적인 디지털 직렬 통신 방법 중 하나이다. 단, 데이터 포맷을 제외한 I2C와 가장 큰 차이점이 2가지가 있는데 각각 다음과 같다. I2C는 모든 슬레이브가 마스터의 클럭과 동기화되어있다는 점이다. 클럭에 의하여 2개의 버스선 중 하나의 버스선으로만 데이터가 송수신되므로 단방향 통신인데에 반해, UART의 경우 통신하는 디바이스 간의 동기화 클럭이 존재하지않아 2개의 버스선 모두 데이터 송수신에 이용될 수 있어 양방향 통신이 가능하며 디바이스간 포트 연결은 그림 8과 같다. 하지만 이를 다른 말로 하면 UART는 통신할 때 클럭이 아닌 다른 동기화 요소가 존재하며 그것은 각각 통신속도와 프레임 포맷이다. 이에 해당하는 내용은 다음과 같다.

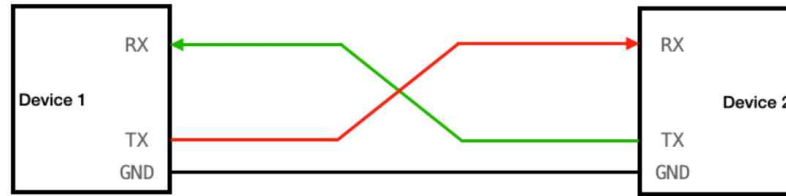


그림 8. UART 통신에서 디바이스간 포트 연결

가. 통신속도(Baud Rate)

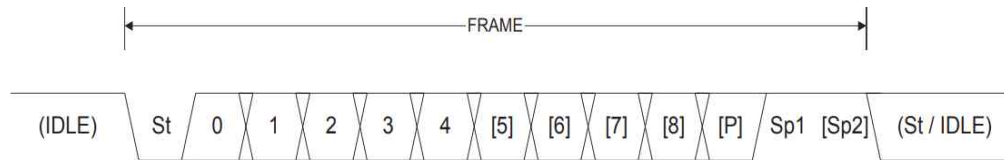
앞서 언급한 바와 같이 UART에서는 동기화 클럭이 존재하지 않기 때문에 통신하는 디바이스간 통신속도를 맞춰줘야한다. ATmega128의 경우 데이터시트에서 그림 9와 같은 통신속도 공식을 제공하고있으며 통신하고자하는 디바이스의 통신속도에 맞게끔 U2X와 UBRR을 설정하면된다. 일반적으로 U2X는 0을 디폴트 값으로 가진다. 예를 들어 그림 9를 참고하여 시스템 클럭이 8MHz이고 U2X를 디폴트 값으로 사용할 때, 통신속도를 9600bps로 맞춰야한다면 UBRR은 51이다.

Operating Mode	Equation for Calculating Baud Rate ⁽¹⁾	Equation for Calculating UBRR Value
Asynchronous Normal Mode (U2X = 0)	$BAUD = \frac{f_{OSC}}{16(UBRR + 1)}$	$UBRR = \frac{f_{OSC}}{16BAUD} - 1$
Asynchronous Double Speed Mode (U2X = 1)	$BAUD = \frac{f_{OSC}}{8(UBRR + 1)}$	$UBRR = \frac{f_{OSC}}{8BAUD} - 1$

그림 9. ATmega128의 UART 통신속도 관련 공식

나. 프레임 포맷

UART의 경우 I2C와 달리 옵션을 이용하여 프레임 포맷을 비교적 자유롭게 설정할 수 있다. UART에서 프레임 포맷의 옵션은 그림 10과 같이 Stop bit를 최대 2개까지 사용할 수 있으며 Parity bit를 사용할 수 있다. 디폴트 프레임 포맷은 Stop bit 1개를 사용하고, Parity bit가 없는 프레임 포맷이다.



- St Start bit, always low.
- (n) Data bits (0 to 8).
- P Parity bit. Can be odd or even.
- Sp Stop bit, always high.
- IDLE No transfers on the communication line (RxD or TxD). An IDLE line must be high.

그림 10. UART 통신에서 가능한 프레임 포맷

3. NMEA(The National Marine Electronics Association) 프로토콜

NMEA 프로토콜은 시간, 위치, 좌표 등 GPS 데이터와 관련된 문장의 문법이다. 기본적인 형식은 그림 11과 같으며 유의미한 데이터는 모두 Checksum range에 포함되어있다. 또한 Address는 문장의 종류를 의미하며, 문장의 종류에 따라 value에 해당하는 부분에 포함된 데이터의 종류와 순서가 다르다. 즉, 자신이 원하는 데이터들이 한 문장 안에 모두 들어있는 최적의 문장을 선택하여 그 문장만 선택적으로 수신하고 분리하는 것이 중요하다. 문장의 종류는 다양하지만 가장 대표적인 문장인 GPGGA, GPRMC를 살펴보자.

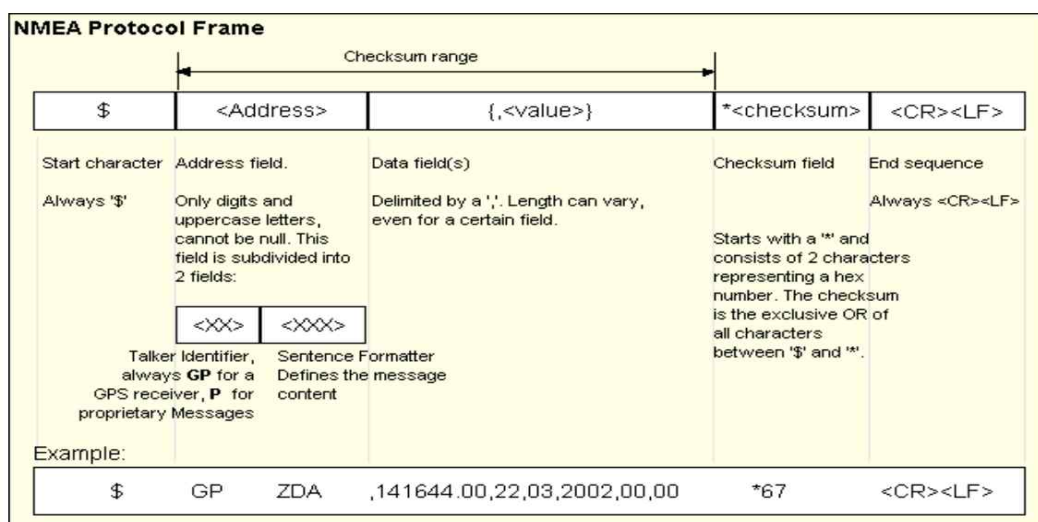


그림 11. NMEA 프로토콜 프레임

가. GPGLA(Global Positioning System Fix Data)

GPGLA 문장은 GPS 수신기의 위치를 고정된 시점에서의 정확한 위치 정보를 제공한다. GPGLA 문장이 갖고있는 유의미한 데이터로는 그림 12와 같이 UTC 시간, 위도, 경도, 고도 등이 있다. 따라서 GPGLA 문장은 이동 경로보다 현재 위치에서의 정확한 위치 정보를 얻는데 용이하다.

Example:

\$GPGLA,092725.00,4717.11399,N,00833.91590,E,1,8,1.01,499.6,M,48.0,M,,0*5B

Field No.	Example	Format	Name	Unit	Description
0	\$GPGLA	string	\$GPGLA	-	Message ID, GGA protocol header
1	092725.00	hhmmss.sss	hhmmss.ss	-	UTC Time, Current time
2	4717.11399	ddmm.mmmmm	Latitude	-	Latitude, Degrees + minutes, see Format description
3	N	character	N	-	N/S Indicator, N=north or S=south
4	00833.91590	dddmm.mmmmm	Longitude	-	Longitude, Degrees + minutes, see Format description
5	E	character	E	-	EW indicator, E=east or W=west
6	1	digit	FS	-	Position Fix Status Indicator, See Table below and Position Fix Flags description
7	8	numeric	NoSV	-	Satellites Used, Range 0 to 12
8	1.01	numeric	HDOP	-	HDOP, Horizontal Dilution of Precision
9	499.6	numeric	msl	m	MSL Altitude
10	M	character	uMsl	-	Units, Meters (fixed field)
11	48.0	numeric	Altref	m	Geoid Separation
12	M	character	uSep	-	Units, Meters (fixed field)
13	-	numeric	DiffAge	s	Age of Differential Corrections, Blank (Null) fields when DGPS is not used
14	0	numeric	DiffStation	-	Diff. Reference Station ID
15	*5B	hexadecimal	cs	-	Checksum
16	-	character	<CR><LF>	-	Carriage Return and Line Feed

그림 12. GPGLA 문장의 형식

나. GPRMC(Recommended Minimum Data)

GPRMC 문장은 GPS 수신기의 최소 위치 정보 및 수신기의 속도와 진행방향에 대한 정보를 제공한다. GPRMC 문장이 갖고있는 유의미한 데이터로는 그림 13과 같이 UTC 시간, 날짜, 위도, 경도, 속도와 진행방향, 자기편각 등이 있다. 따라서 GPRMC 문장은 이동 경로 등의 정보를 얻는데 용이하여 항법 시스템, 내비게이션 등에 사용될 수 있다.

Example:

\$GPRMC,083559.00,A,4717.11437,N,00833.91522,E,0.004,77.52,091202,,,A*57					
Field No.	Example	Format	Name	Unit	Description
0	\$GPRMC	string	\$GPRMC	-	Message ID, RMC protocol header
1	083559.00	hhmmss.sss	hhmmss.ss	-	UTC Time, Time of position fix
2	A	character	Status	-	Status, V = Navigation receiver warning, A = Data valid, see Position Fix Flags description
3	4717.11437	ddmm.mmmm	Latitude	-	Latitude, Degrees + minutes, see Format description
4	N	character	N	-	N/S Indicator, hemisphere N=north or S=south
5	00833.91522	dddmm.mmmm	Longitude	-	Longitude, Degrees + minutes, see Format description
6	E	character	E	-	E/W indicator, E=east or W=west
7	0.004	numeric	Spd	knots	Speed over ground
8	77.52	numeric	Cog	degrees	Course over ground
9	091202	ddmmyy	date	-	Date in day, month, year format
10	-	numeric	mv	degrees	Magnetic variation value, not being output by receiver
11	-	character	mvE	-	Magnetic variation E/W indicator, not being output by receiver
12	-	character	mode	-	Mode Indicator, see Position Fix Flags description
13	*57	hexadecimal	cs	-	Checksum
14	-	character	<CR><LF>	-	Carriage Return and Line Feed

그림 13. GPRMC 문장의 형식

4. 지구 자기장을 이용한 방위각(Heading)

앞서 살펴본 절에 의하면 GPS 수신기가 GPRMC 문장을 수신할 수 있다면 그것만을 이용해서도 진행방향에 대한 방위각을 얻을 수 있다. 하지만 GPS 수신기가 위성에서 송신한 GPRMC 문장을 온전히 수신할 수 있다는 보장이 없다. 따라서 위성 통신에 의존하지 않고 방위각을 구할 수 있는 방법이 필요하며 대표적인 방법이 지구 자기장을 이용하는 것이다.

가. 자북(Magnetic North)과 진북(True North)

지구의 북쪽은 2가지가 있으며 그것은 각각 자북과 진북이다. 그림 14와 같이 자북은 지구 자기장에 의해 나침반이 가리키는 북쪽을 의미하며, 진북은 지리학적 북쪽을 의미한다. 참고로 진북은 자전축의 북극과 일치한다. 우리가 시스템을 만들 때 필요한 것은 진북을 기준으로한 방위각인데 진북과 자북은 지역마다 약간의 차이가 존재한다. 이러한 차이를 자기편각(Magnetic Declination Angle)이라고 하며 자기편각을 얻을 수 있는 방법은 GPS 신호 수신외에도 2가지가 더 있다. 하나는 자기편각을 알려주는 웹사이트^[1]를 방문하는 것과 다른 하나는 NOAA(National Oceanic and Atmospheric Administration)에서 공개한 WMM(World Magnetic Model)^[2] COF 파일과 소프트웨어를 사용하는 것이다. 하지만 이와같은 방법은 8비트 MCU에서 동작하기 어려우므로 생략한다. 따라서 시

[1] <https://www.magnetic-declination.com/>

[2] <https://www.ncei.noaa.gov/products/world-magnetic-model>

시스템을 구현할 때는 현재 자신의 지역에서의 자기편각을 알고있다는 특수한 상황을 가정하도록 한다.

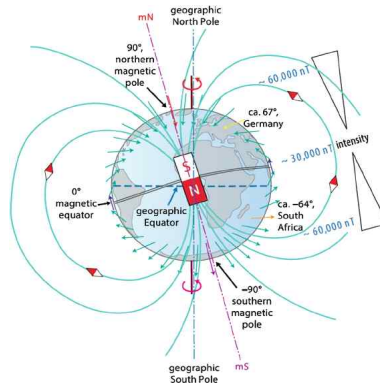


그림 14. 자북과 진북 비교

나. 지구 자기장을 이용한 방위각 계산

자기장 센서에 의해 측정된 지구 자기장은 3차원 벡터로 표현될 수 있으며 그 벡터는 그림 15에서 $\mathbf{H}_{\text{Earth}}$ 와 같다. 이 벡터를 나침반과 같이 사용하려면 $\mathbf{H}_{\text{Earth}}$ 을 XY 평면에 사영시켜야 하며 그렇게 사영된 벡터는 $\mathbf{H}_{\text{Nord}} = (H_{\text{Nord}(x)}, H_{\text{Nord}(y)}, 0)$ 와 같다. X축을 자북의 방향이라고 가정할 때 X축을 0° , Y축을 90° 로 생각할 수 있다. 이러한 기준을 이용하여 $\mathbf{H}_{\text{Nord}}$ 이 자북으로부터 얼마나 기울어져있는지 수식으로 $\phi = \tan^{-1}(\frac{H_{\text{Nord}(y)}}{H_{\text{Nord}(x)}})$ 와 같이 표현할 수 있다. 따라서 진북을 기준으로한 방위각의 수식은 위의 계산 결과에 자기편각만큼 보정한 $\text{Heading} = \phi + \text{MDA}$ 이다.

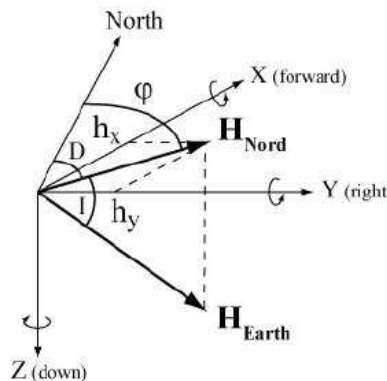


그림 15. 직교 좌표계에서
지구 자기장의 표현

5. 지구 위에서 두 좌표 사이의 거리와 방위각(Bearing)

지구 위에서 한 좌표에서 다른 좌표에 도달하는 방법은 두 좌표 사이의 거리만큼 방위각을 따라 이동하는 것이다. 여기서의 방위각은 진북을 기준으로 현재 위치에서 목표 지점까지의 접선이 얼마나 기울어져 있는지를 의미하므로, 이전 절에서 설명한 지구 자기장에 의한 방위각과 다르다. 따라서 두 좌표 사이의 거리가 짧을 때 평면 지도에서 수직선으로부터 두 위치 사이의 직선이 얼마나 기울어져 있는지로 근사될 수 있다. 이러한 거리와 방위각은 지구를 어떤 모델로 해석하느냐에 따라 차이가 있으며 크게 두 가지로 분류할 수 있다. 그 내용은 다음과 같다.

가. 하버사인(Haversine)의 공식

지구를 가장 간단한 모델로 해석하는 방법은 지구를 반지름 6371km인 완전한 구로 보는 것이다. 이렇게 지구를 완전한 구로 볼 때 거리와 방위각을 계산하는 공식을 하버사인의 공식이라고 하며 그 공식은 그림 16과 그림 17과 같다. 하버사인의 공식은 지구를 타원체로 해석하여 거리와 방위각을 구하는 공식보다 간단하지만, 지구는 타원체이기 때문에 하버사인의 공식은 두 GPS 좌표 사이의 거리가 멀수록 오차가 커지는 문제가 발생한다. 따라서 이번에는 지구를 타원체로 해석하여 거리와 방위각을 구하는 공식을 살펴보자.

$$\text{Haversine } a = \sin^2(\Delta\phi/2) + \cos \phi_1 \cdot \cos \phi_2 \cdot \sin^2(\Delta\lambda/2)$$

$$\text{formula: } c = 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a})$$

$$d = R \cdot c$$

where ϕ is latitude, λ is longitude, R is earth's radius (mean radius = 6,371km);
note that angles need to be in radians to pass to trig functions!

그림 16. 하버사인의 거리 공식

$$\text{Formula: } \theta = \text{atan2}(\sin \Delta\lambda \cdot \cos \phi_2, \cos \phi_1 \cdot \sin \phi_2 - \sin \phi_1 \cdot \cos \phi_2 \cdot \cos \Delta\lambda)$$

where ϕ_1, λ_1 is the start point, ϕ_2, λ_2 the end point ($\Delta\lambda$ is the difference in longitude)

그림 17. 하버사인의 방위각 공식

나. 빈센티(Vincenty)의 공식

하버사인의 공식과 반대로 지구를 타원체로 볼 때 거리와 방위각을 계산하는 공식을 빈센티의 공식이라고 하며 그림 19와 같다. s 는 두 GPS 좌표 사이의 거리, α_1

은 첫 번째 좌표로부터 두 번째 좌표까지의 방위각, 반대로 α_2 는 두 번째 좌표로부터 첫 번째 좌표까지의 방위각이다. 빈센티의 공식은 모든 상황에서 하버사인의 공식보다 정밀도가 높지만 공식이 복잡하다는 단점이 있다. 공식을 사용하기에 앞서 지구의 장반경과 단반경에 대한 국제 표준을 살펴보자. 역사적으로 지구의 장반경과 단반경에 대한 국제 표준은 계속 변동되고 있으며 현재는 WGS 84라는 모델을 사용하고 있다. 또한 위성은 WGS 84를 기준으로 좌표를 계산하기 때문에 해당 시스템에서도 WGS 84를 사용할 것이다. WGS 84에 대한 규격은 그림 18과 같다. 하버사인의 공식과 빈센티의 공식에 대하여 자세한 수치 데이터와 오차는 IV 장에서 다루도록한다.

Ellipsoid reference	Semi-major axis a	Semi-minor axis b	Inverse flattening $1/f$
GRS 80	6 378 137.0 m	$\approx$ 6 356 752.314 140 m	298.257 222 100 882 711...
WGS 84 ^[6]	6 378 137.0 m	$\approx$ 6 356 752.314 245 m	298.257 223 563

그림 18. WGS 84의 규격

Given the coordinates of the two points (Φ_1, L_1) and (Φ_2, L_2) , the inverse problem finds the azimuths α_1, α_2 and the ellipsoidal distance s .

Calculate U_1, U_2 and L , and set initial value of $\lambda = L$. Then iteratively evaluate the following equations until λ converges:

$$\begin{aligned}\sin \sigma &= \sqrt{(\cos U_2 \sin \lambda)^2 + (\cos U_1 \sin U_2 - \sin U_1 \cos U_2 \cos \lambda)^2} \\ \cos \sigma &= \sin U_1 \sin U_2 + \cos U_1 \cos U_2 \cos \lambda \\ \sigma &= \arctan2(\sin \sigma, \cos \sigma)^{[1]} \\ \sin \alpha &= \frac{\cos U_1 \cos U_2 \sin \lambda}{\sin \sigma}^{[2]} \\ \cos^2 \alpha &= 1 - \sin^2 \alpha \\ \cos(2\sigma_m) &= \cos \sigma - \frac{2 \sin U_1 \sin U_2}{\cos^2 \alpha} = \cos \sigma - \frac{2 \sin U_1 \sin U_2}{1 - \sin^2 \alpha}^{[3]} \\ C &= \frac{f}{16} \cos^2 \alpha [4 + f(4 - 3 \cos^2 \alpha)] \\ \lambda &= L + (1 - C)f \sin \alpha \{ \sigma + C \sin \sigma [\cos(2\sigma_m) + C \cos \sigma (-1 + 2 \cos^2(2\sigma_m))] \}\end{aligned}$$

When λ has converged to the desired degree of accuracy (10^{-12} corresponds to approximately 0.06 mm), evaluate the following:

$$\begin{aligned}u^2 &= \cos^2 \alpha \left(\frac{a^2 - b^2}{b^2} \right) \\ A &= 1 + \frac{u^2}{16384} (4096 + u^2 [-768 + u^2 (320 - 175u^2)]) \\ B &= \frac{u^2}{1024} (256 + u^2 [-128 + u^2 (74 - 47u^2)]) \\ \Delta \sigma &= B \sin \sigma \left\{ \cos(2\sigma_m) + \frac{1}{4} B \left(\cos \sigma [-1 + 2 \cos^2(2\sigma_m)] - \frac{1}{6} B \cos[2\sigma_m] [-3 + 4 \sin^2 \sigma] [-3 + 4 \cos^2(2\sigma_m)] \right) \right\} \\ s &= bA(\sigma - \Delta \sigma) \\ \alpha_1 &= \arctan2(\cos U_2 \sin \lambda, \cos U_1 \sin U_2 - \sin U_1 \cos U_2 \cos \lambda) \\ \alpha_2 &= \arctan2(\cos U_1 \sin \lambda, -\sin U_1 \cos U_2 + \cos U_1 \sin U_2 \cos \lambda)\end{aligned}$$

Between two nearly antipodal points, the iterative formula may fail to converge; this will occur when the first guess at λ as computed by the equation above is greater than π in absolute value.

그림 19. 빈센티의 공식

III. 시스템 구현

1. 하드웨어 구현

II 장에서는 해당 시스템에 사용될 이론과 기술에 대하여 살펴보았다. III 장에서는 시스템을 실제로 구현하기 위한 방법에 대하여 살펴볼 것이다. 이번 장은 하드웨어 구현, 소프트웨어 구현으로 나눌 수 있다. 해당 절은 하드웨어 구현을 다룰 것이며 대부분의 부품은 모듈화된 디바이스를 사용할 수 있기 때문에 블록도로 간단하게 표현될 수 있다. 따라서 시스템 블록도를 통해 ATmega128로 부품별 제어 방법을 먼저 살펴본 후 블록도와 같이 시스템을 구현하기 위한 ATmega128의 포트 할당을 살펴본다. 이후 포트에 할당된 키 매트릭스와 LED를 정상적으로 동작시키기 위한 보조 회로도를 살펴본다.

가. 시스템 블록도

그림 20은 전체 시스템 블록도를 표현한 것이다. 우선 ATmega128 기준 좌측의 블록을 먼저 살펴보자.

키 매트릭스는 인터럽트 방식으로 제어가 불가능하기 때문에 GPIO를 이용한 폴링(Polling) 방식을 이용해야 한다. HMC5883L은 보편적인 자기장 센서 중 하나이며 I2C 통신을 이용하여 제어가 가능하다. 참고로 자기장은 벡터이기 때문에 HMC5883L이 측정하는 것은 지구의 자기장뿐만 아니라 주변 회로의 전류에 의한 자기장까지 포함되어있다. 따라서 HMC5883L은 주변 부품들과 충분히 떨어져있는 곳에 배치하거나 전류에 의한 자기장을 차폐시킬 수 있는 투자율이 높은 물질로 HMC5883L의 주변 회로를 감싸야한다. NEO-6M은 보편적인 GPS 모듈 중 하나이며 수신된 내용을 UART 통신을 통해 ATmega128에 송신할 수 있다. ATmega128 기준 좌측의 블록들은 모두 살펴보았으므로 우측의 블록을 살펴보자.

해당 시스템은 키 매트릭스의 #버튼을 이용하여 음수 입력과 커맨드 입력을 제어하기 때문에 각각의 상황에 따른 표시 방법이 필요하다. 그 방법을 LED로 선택하였기 때문에 LED 2개를 블록도에 표현하였으며 각각의 LED는 GPIO로 동작한다. 해당 시스템은 사용자와 상호작용해야 하므로 시스템의 현재 상태를 글자와 그림으로 표현할 수 있어야 한다. 따라서 글자와 그림을 표현하기 위해 캐릭터 LCD와 OLED 디스플레이를 사용할 것이다. 캐릭터 LCD는 GPIO 혹은 I2C 모듈인 PCF8574를 추가적으로 사용하여 I2C 통신으로 제어할 수 있는데 해당 시스템은 후자의 방법을 선택했다. 그림을 표현하기 위한 OLED 디스플레이는 SSD1309를

선택하였으며 I2C 통신을 통해 제어할 수 있다.

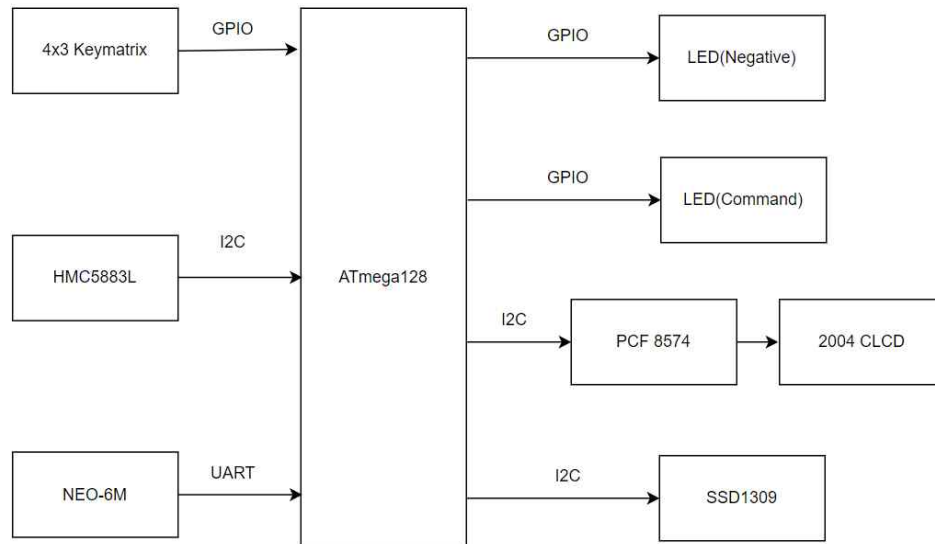


그림 20. 전체 시스템 블록도

나. 포트할당

이번 절에서는 부품별로 ATmega128의 포트를 할당하는 방법에 대하여 알아볼 것이다. 해당 시스템에서 사용할 부품은 크게 두가지로 분류할 수 있는데 각각 사용자 인터페이스용 부품과 사용자 인터페이스용이 아닌 부품이며 후자는 다시 자기장 센서와 GPS 모듈로 분류될 수 있다. 사용자 인터페이스용 부품의 포트할당은 표 1, 자기장 센서 및 GPS 모듈의 포트할당은 표 2와 같다.

표 1. 사용자 인터페이스 포트할당

포트	명칭	기능	용도
PA0, PA1, PA2, PA3, PA4, PA5, PA6	AD0~AD7	GPIO INPUT, GPIO OUTPUT	키 매트릭스 제어
PG0, PG1	$\overline{WR}, \overline{RD}$	GPIO OUTPUT	음수 LED, 커맨드 LED 제어
PD0, PD1	SCL/INT0, SDA/INT1	I2C	PCF8574, SSD1309 제어

표 2. 자기장 센서 및 GPS 모듈 포트할당

포트	명칭	기능	용도
PD0, PD1	SCL/INT0, SDA/INT1	I2C	HMC5883L 제어
PE0, PE1	RXD0/PDI, TXD0/PDO	UART	NEO-6M 제어

다. 보조 회로도

이번 절에서는 키 매트릭스와 LED 저항 값과 저항 연결 방법에 대하여 살펴볼 것이며 보조 회로도는 그림 21과 같다. 키 매트릭스는 12개의 스위치로 이루어져 있으며 스위치가 눌렸는지 검출하기 위해 저항을 연결하는 방법은 두 가지가 존재한다. 각각 풀업(Pull-up), 풀다운(Pull-down)이라고 하며 해당 시스템은 풀다운을 사용했다. 저항값에 대해서는 사용하는 키 매트릭스 제품의 데이터시트를 참고하여 접촉저항보다 충분히 큰 값으로 설정한다. 해당 시스템에서 사용하는 키 매트릭스의 데이터시트^[3]에 의하면 접촉저항이 200ohm이 존재하며 최대 전류는 20mA이다. 따라서 풀다운 저항은 200ohm보다 충분히 커야하며, 5V에 대하여 전류가 20mA보다 작아야한다. 해당 시스템에서는 10kohm을 사용하였다.

LED에 연결할 저항 또한 LED에 흐르는 최대 전류를 제한하는 용도로 사용한다. LED의 종류와 색상 별로 순방향 바이어스 전압과 최대 전류가 조금씩 다르지만, 전자회로 실험에서 사용하는 LED는 보통 순방향 바이어스 전압을 2V, 최대 전류를 10mA로 생각하고 저항값을 설정해줘도 된다. 5V 입력에 대하여 저항을 330ohm으로 설정해주면 LED에 흐르는 전류를 10mA 미만으로 설정해줄 수 있다.

[3] NTREX, "NT-804AN-BW"

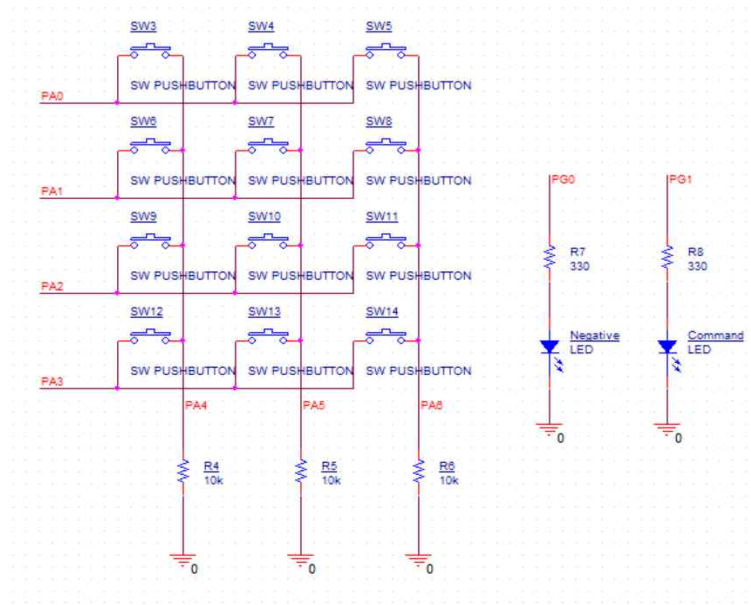


그림 21. 키 매트릭스와 LED에 대한 보조 회로도

2. 소프트웨어 구현

소프트웨어 구현은 해당 시스템에서 중요한 기능을 하는 부품들에 대한 순서도를 포함한다. 해당 시스템을 사용자가 직접적으로 제어할 수 있는 유일한 인터페이스 부품은 키 매트릭스이다. 따라서 처음에는 키 매트릭스를 이용하여 숫자를 입력하는 방식과 양수와 음수를 구분하는 방법에 대하여 알아볼 것이다. 그리고 사용자는 2가지 지도 그래픽을 통해 자신의 현재 위치와 목표 위치 사이의 관계를 쉽게 파악할 수 있어야한다. 따라서 사용자가 OLED 디스플레이에 출력될 지도의 그래픽을 선택하는 방법에 대하여 알아볼 것이다. 그 다음에는 HMC5883L이 측정한 자기장의 세기를 읽어서 활용하는 방법 그리고 NEO-6M이 수신한 GPRMC 문장을 활용하는 방법에 대하여 살펴볼 것이다. 마지막에는 모든 부품들이 함께 동작하여 하나의 시스템으로서 동작시킬 수 있는 방법에 대하여 알아볼 것이다. 소프트웨어 구현에 대한 소스코드는 저자의 깃허브^[4]를 참고하라.

가. 4x3 Key Matrix

이제 키 매트릭스를 이용하여 n개의 숫자를 입력받는 방법에 대하여 알아볼 것이다. 그 방법은 키 매트릭스 입력에 대하여 다음과 같은 3가지 규칙을 정하는 것이다.

[4] https://github.com/snlee99/ATmega128_Navigation.git

1. n개의 숫자를 입력받아야 하는데 사용자가 n개보다 적은 숫자를 입력하고 다음 모드로 넘어가려하면 처음부터 다시 숫자를 입력받는다.
2. 다음 모드로 넘어가려할 때 사용자는 * 버튼을 눌러야한다.
3. 음수 입력과 커맨드는 # 버튼으로 제어한다.

이와 같은 규칙을 순서도로 표현하면 그림 22와 같아진다. 세 번째 규칙에 대하여 추가적인 설명을 하자면 음수와 양수의 구분을 BOOL 형식의 변수 `flag_negative`를 두어 `flag_negative`이 true일 때 입력한 숫자는 음수로 입력되며 false일 때는 양수로 입력된다. 디폴트값은 false이다. 또한 세 번째 규칙 중 커맨드에 대한 내용은 순서도에 표현되어있지 않은데 그 이유는 순서도보다는 글로 이해하는 것이 더 빠를 것이라 생각하기 때문이다. 커맨드는 해당 장의 마 절 그림 32 통합 알고리즘의 순서도 기준 입력 대기 상태와 추적 모드에서만 활성화되어있으며 편의상 n번째 커맨드 입력을 `CMDn`으로 표현하도록 하자. `CMDn`을 입력하는 방법은 # 버튼을 누른 후 숫자 n을 입력하면 자동으로 실행된다. 참고로 커맨드는 총 3가지 존재하며 각각 자기편각 수정(`CMD1`), 지도 그래픽 변경(`CMD2`) 그리고 거리와 방위각 계산 공식 변경(`CMD3`)이다.

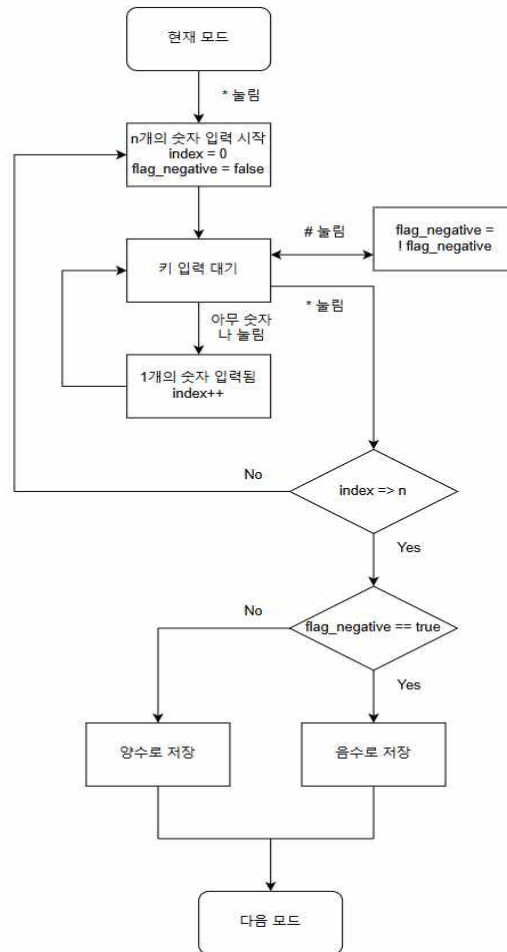


그림 22. 키 매트릭스 응용 순서도

나. SSD1309 OLED Display

OLED 디스플레이의 순서도를 논하기에 앞서 OLED 디스플레이에 표시할 사진에 대한 배열을 손쉽게 얻는 방법을 먼저 알아보자. 해당 시스템에서 사용하는 OLED 디스플레이는 128x64이며 픽셀 1개의 Off와 On의 기준을 각각 0과 1이라고 하자. 만약 배열의 형식을 unsigned char로 할 때 1개의 원소가 표현할 수 있는 픽셀의 수는 최대 8개이다. 따라서 배열의 크기는 1024이 되는데 현실적으로 사람이 1024개의 unsigned char 형식 숫자를 정확하게 작성하는 것은 불가능하다. 하지만 소프트웨어를 이용하면 사진에 해당하는 배열을 쉽게 얻을 수 있으며 그 방법은 사진을 BMP 파일로 변경한 후 LCD Assistant^[5]라는 소프트웨어를 사용하는 것이다. 참고로 그림 23과 그림 24는 해당 시스템에서 사용하는 지도 그래픽의 원본이다.

[5] http://en.radzio.dxp.pl/bitmap_converter/



그림 23. 세계지도 원본



그림 24. 한반도 지도 원본

이번에는 위의 두 사진을 CMD2로 변경하는 방법에 대하여 알아보자. 해당 시스템의 모든 커맨드는 음수 입력과 유사한 방식으로 제어된다. BOOL 형식의 변수 flag_korea를 두어 flag_korea이 true이면 그래픽을 한반도로 하며 false이면 그래픽을 세계지도로 한다. 디폴트값은 false이며 CMD2이 입력될 때 마다 flag_korea를 반전시킬 수 있다. 이에 대한 내용을 순서도로 표현할 때 그림 25과 같이 된다. 그림 25에 따라 전환되는 OLED 디스플레이의 배경은 각각 그림 26, 27와 같게된다.

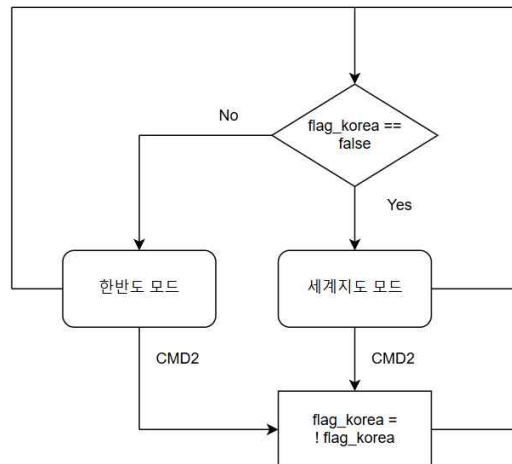


그림 25. SSD1309 응용 순서도

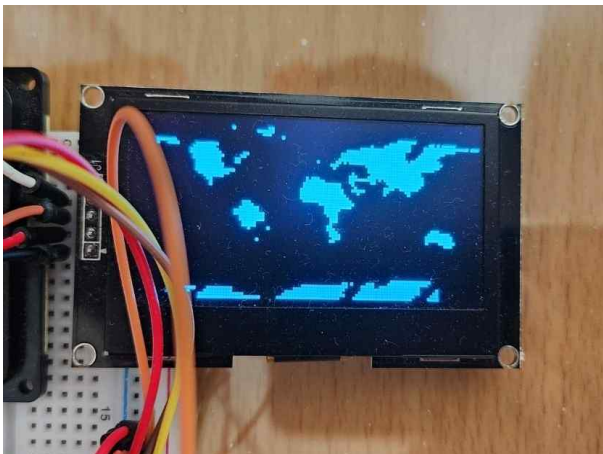


그림 26. 세계지도 출력 테스트

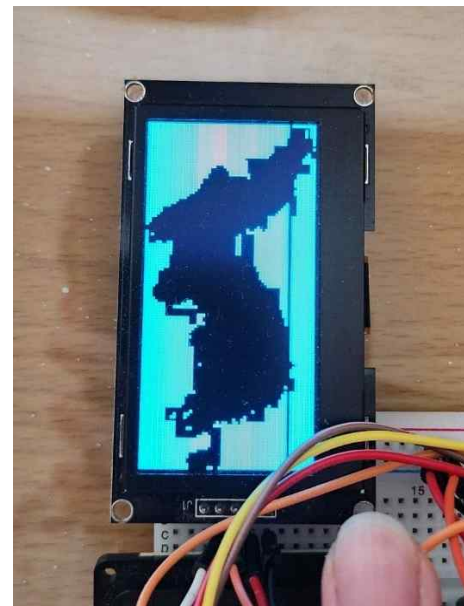


그림 27. 한반도 출력 테스트

위의 그림에서 보다시피 세계지도는 OLED 디스플레이를 가로로 사용하고, 한반도는 세로로 사용한다는 것을 알 수 있으며, 화면의 경계에 해당하는 좌표가 서로 다르다. 화면의 좌측 상단을 원점으로 둘 때, 화면의 경계에 대한 좌표는 표 3와 같다. 또한 OLED 디스플레이 상에서 위치를 수평선과 수직선의 교차점으로 표현할 때, 세계지도 모드와 한반도 모드에서의 수평선과 수직선의 공식은 표 4, 5와 같다.

표 3. OLED 디스플레이 위치에 따른 GPS 좌표

	$x = 0$	$x = \text{Width}$	$y = 0$	$y = \text{Height}$
세계지도 모드	180 ° W	180 ° E	90 ° N	90 ° S
한반도 모드	33 ° N	43 ° N	124 ° E	131 ° E

표 4. GPS 좌표에 따른 OLED 디스플레이의 수평선 위치 공식

	수평선 위치 공식
세계지도 모드	$y = \text{Height} - \left\{ \frac{\text{Latitude}}{90} \times \frac{\text{Height}}{2} + \frac{\text{Height}}{2} \right\}$
한반도 모드	$x = \text{Width} \times \frac{\text{Latitude} - \min(\text{Latitude of korea})}{\max(\text{Latitude of korea}) - \min(\text{Latitude of korea})}$

표 5. GPS 좌표에 따른 OLED 디스플레이의 수직선 위치 공식

	수직선 위치 공식
세계지도 모드	$x = \frac{\text{Longitude}}{180} \times \frac{\text{Width}}{2} + \frac{\text{Width}}{2}$
한반도 모드	$y = \text{Height} \times \frac{\text{Longitude} - \min(\text{Longitude of korea})}{\max(\text{Longitude of korea}) - \min(\text{Longitude of korea})}$

다. HMC5883L

이번에는 HMC5883L이 측정한 자기장을 진북 기준의 방위각으로 변환하는 방법에 대하여 알아볼 것이다. 우선 그림 28을 먼저 보도록 하자. 초기화를 제외한 처음의 블록 3개는 II 장 4 절에서 논한 공식과 정확히 일치한다. 하지만 그 다음의 내용은 기존에 논의한 적 없는 내용인데, 이것은 방위각에 자기편각을 더한 후 각도가 360°를 초과하거나 0°보다 작아졌을 때 헤딩을 0° ~ 360°사이의 값으로 보정하는 방법에 대한 내용이다.

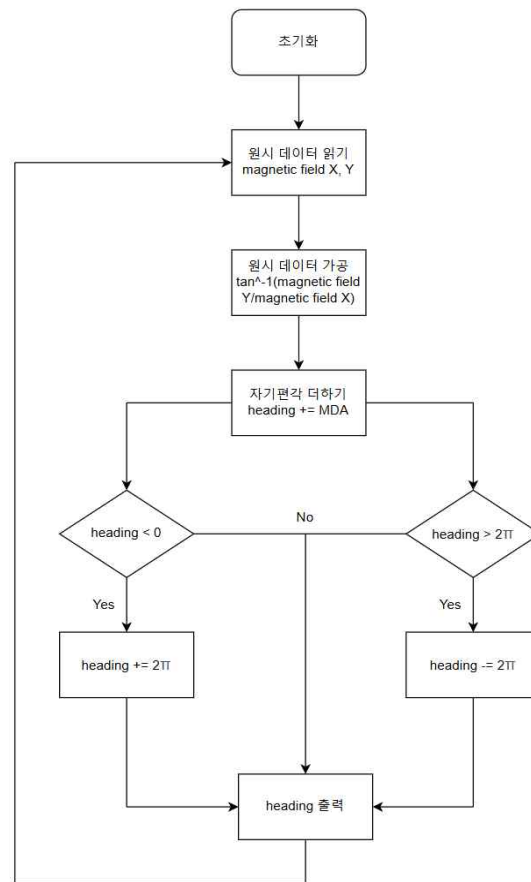


그림 28. HMC5883L 응용 순서도

그림 29는 자기편각을 0° 로 둔 후 자북 기준의 방위각을 측정했을 때의 실험 사진이다. 나침반은 대략 293° 를 가리키고있지만, 자기장 센서는 283° 를 측정했다. 이 현상은 나침반의 자석에 의해 자기장 센서가 측정하는 지구의 자기장이 간섭되었기 때문이다. 나침반에 대한 자기장 센서의 오차는 대략 $\pm 10^\circ$ 이며 자기장 센서의 측정값은 오차범위 안에 있다.

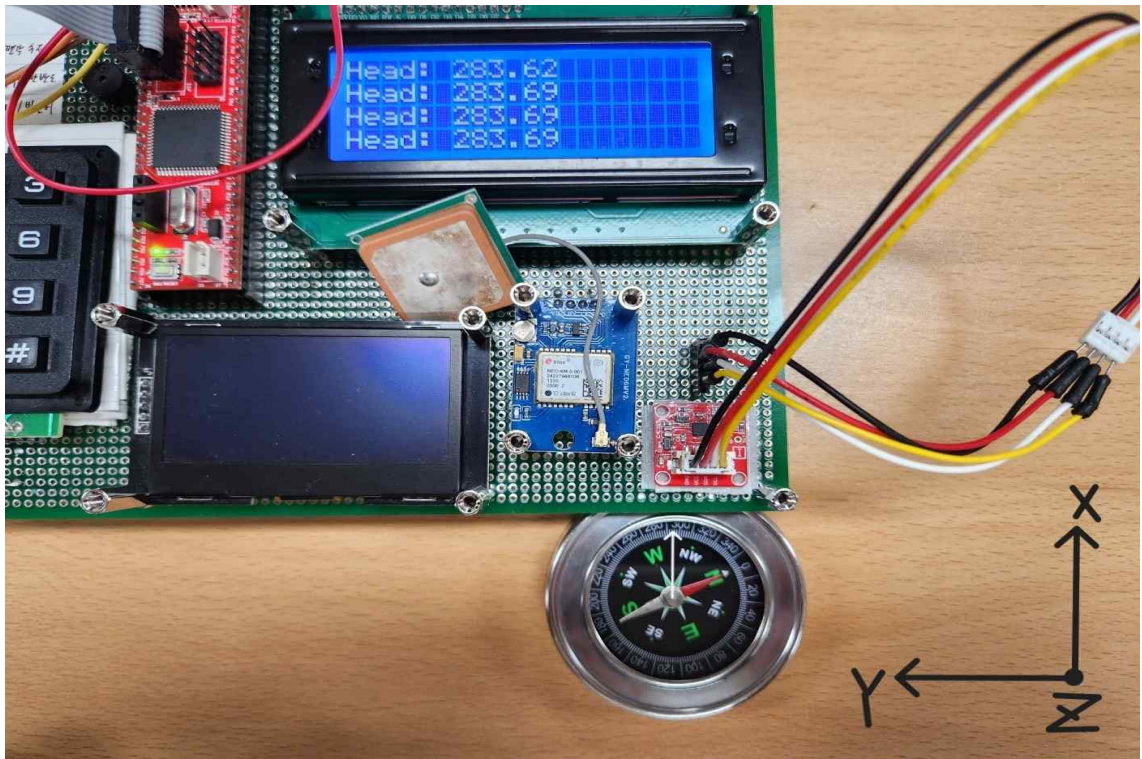


그림 29. 나침반과 자기장 센서의 측정값 비교

라. NEO-6M

해당 시스템에서 사용자가 GPRMC 문장이 필요한 상황은 두 가지밖에 없다. 그것은 각각 바로 다음 절의 그림 32 통합 알고리즘 순서도 기준 대기 상태와 추적 모드이다. 따라서 GPRMC 문장을 가공하는 방식은 두 가지라 할 수 있다. 대기 상태에서는 GPRMC 문장의 UTC 날짜/시간, 좌표 데이터를 분리하여 캐릭터 LCD에 그대로 출력하되 만약 flag_korea이 true이면 UTC 날짜/시간을 한국 표준시인 UTC+9로 변환하여 출력하도록 한다. 만약 UTC 날짜가 2월일 경우 다음 날짜로 넘어갈 때 날짜가 정상적으로 변하도록 윤년을 체크한다. 추적 모드에서는 GPRMC 문장의 좌표 데이터만 분리하여 사용자가 입력한 좌표 사이의 거리와 방위각을 계산하여 캐릭터 LCD에 출력하도록 한다. 이와같은 내용을 순서도로 표현하면 그림 30과 같다.

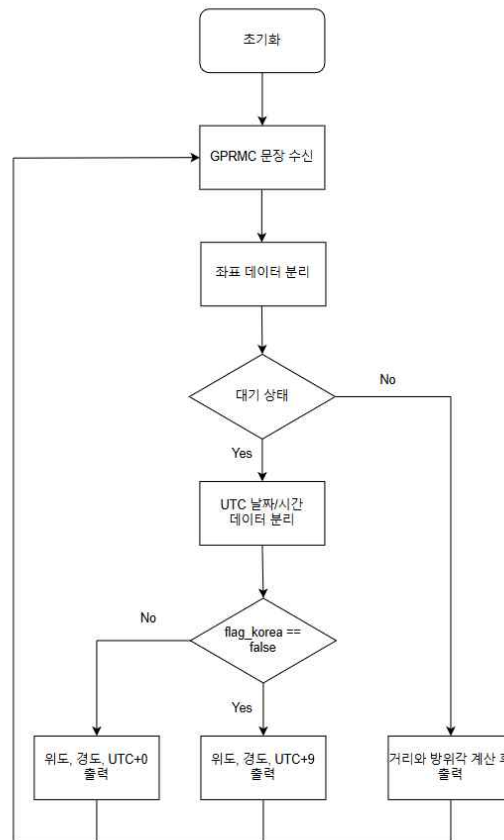


그림 30. NEO-6M 응용 순서도

그림 31은 NEO-6M을 이용해 NMEA 프로토콜에 따른 작성된 GPRMC 문장이 출력되고있는 모습을 볼 수 있다. 비록 속도와 자기편각을 수신하는데에는 실패했지만 나머지 정보는 모두 온전하므로, 방금 전에 논한 자기장 센서를 추가하여 보완하면 된다.

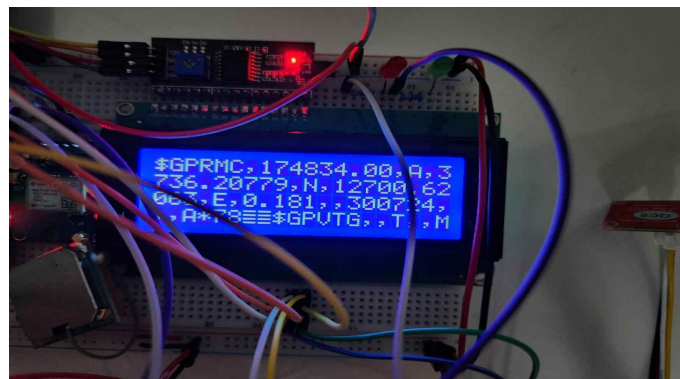


그림 31. NEO-6M 동작 테스트

마. 통합 알고리즘

이번에는 해당 시스템의 통합 알고리즘에서 살펴볼 것이다. 그림 32에서 초기화를 제외한 모서리가 곡선인 사각형 블록이 실제로 사용자가 키 매트릭스로 상호작용할 수 있는 부분이며 대기 상태를 먼저 살펴보도록 하자. 대기 상태의 주 기능은 사용자에게 현재 날짜/시간 그리고 자신의 좌표를 캐릭터 LCD로 알려주는 것이다. 추가적으로 OLED 디스플레이에도 사용자의 위치가 수직선과 수평선의 교점으로 표현된다. 그림 32에 따르면 대기 상태에서 3가지 방향으로 갈 수 있는데 CMD1이 입력되면 자기편각 입력 모드로 진입한다. 자기편각 입력 모드에서는 업데이트할 자기편각 용 숫자 4개를 입력할 수 있다. 단위에 대해서는 표현되어있지 않지만 $MDA = \pm dd^{\circ}mm'$ 를 염두에 두고 4개의 숫자를 입력한 후 * 버튼을 누르면 자기편각이 수정되며 다시 대기 상태로 돌아간다. CMD2이 입력되면 flag_korea를 반전시켜 OLED 디스플레이에 출력되는 지도의 그래픽을 변경할 수 있다. 대기 상태에서 * 버튼을 누르면 추적 모드로 진입하기 전 목적지의 GPS 좌표를 입력하는 모드로 진입한다. 위도 입력 모드에서는 7개의 숫자, 경도 입력 모드에서는 8개의 숫자를 입력하게 되어있다. 자기편각 입력 모드와 같이 위도와 경도 입력 모드에서도 소수점이 표현되어있지는 않지만, 둘 다 소수점 다섯 번째 자리까지 $^{\circ}$ 단위로 입력한다는 점을 염두에 두고 입력하면 된다. 위도와 경도가 모두 정상적으로 입력되었을 때 추적 모드로 진입한다. 추적 모드의 주 기능은 실시간으로 업데이트되는 현재 위치를 기준으로 사용자가 입력한 목적지 사이의 거리와 방위각을 계산하여 캐릭터 LCD에 자기장에 의한 방위각과 함께 사용자에게 알려준다. 추적 모드에서는 4가지 방향으로 갈 수 있는데, CMD1과 CMD2는 대기 상태와 동일하지만, CMD3은 추적 모드에서만 사용할 수 있는 커맨드이다. CMD3은 해당 시스템의 핵심 기능인 거리와 방위각 계산 공식을 사용자가 직접 선택할 수 있도록 하는 것인데 그 방법은 그림 33과 같다. 이번에도 CMD3이 입력될 때 마다 BOOL 형식의 변수 flag_vincenty를 반전시켜 거리와 방위각을 계산할 때 사용자가 하버사인의 공식과 빈센티의 공식 중 어느 공식을 사용할지 선택할 수 있다. flag_vincenty이 true일 때는 빈센티의 공식을 사용하고 false일 때는 하버사인의 공식을 사용하며 디폴트값은 false이다. 사용자가 추적 모드를 종료하고 싶을 때는 * 버튼을 눌러서 다시 대기 상태로 진입할 수 있다.

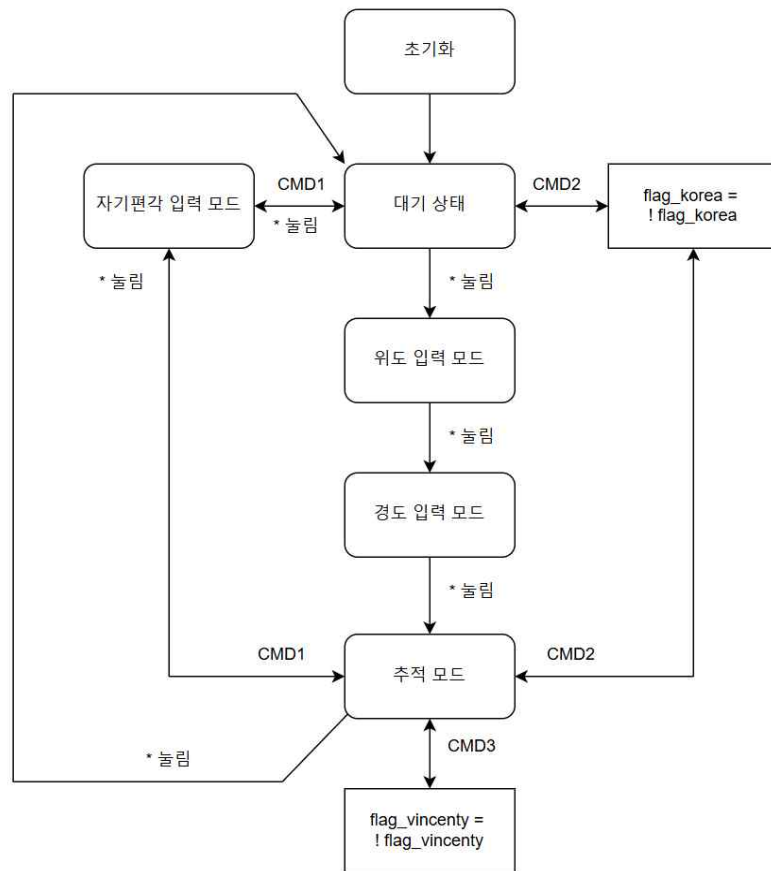


그림 32. 통합 알고리즘 순서도

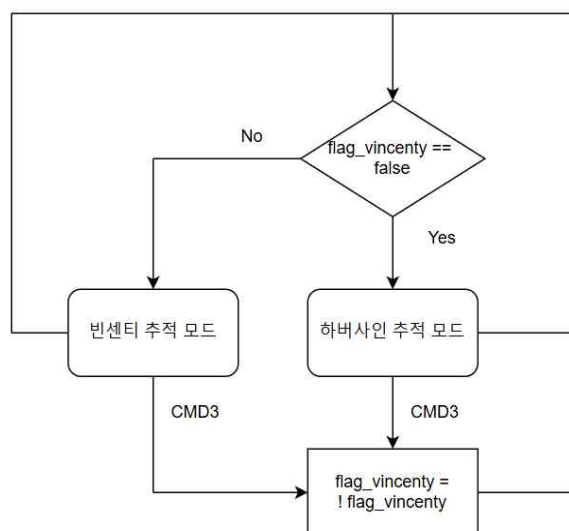


그림 33. 공식 변환 순서도

IV. 실험 및 결과

1. 기준좌표(신한대학교)

이번 장에서는 III 장의 방식으로 만들어진 시스템의 결과에 대한 수치 데이터를 논할 것이다. 수치 데이터에 대하여 논하기에 앞서 신한대학교에서 시스템의 대기 상태를 촬영한 사진 그림 34, 그림 35를 확인해보자. 두 사진의 위도와 경도를 정리하여 평균값을 구하면 표 6과 같다. 또한 두 좌표 사이의 거리를 구글지도를 이용하여 거리를 구하면 그림 36과 같이 17.89m가 차이나며 두 좌표 모두 사진 촬영 장소인 신흥대학공학관과 매우 가까이에 위치한다. 2 절의 실험에서 사용할 기준좌표로 두 신한대학교 좌표의 평균값을 사용할 것이다.



그림 34. 세계지도 모드 출력

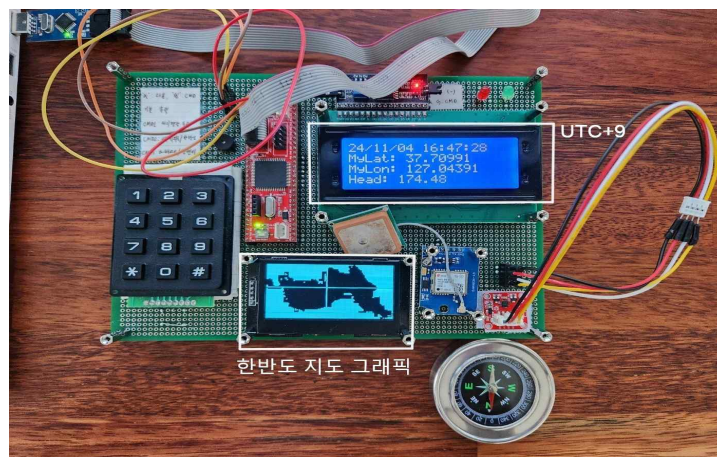


그림 35. 한반도 모드 출력

표 6. 기준좌표(신한대학교) 측정

	위도	경도
세계지도 모드 출력	37.70986	127.04410
한반도 모드 출력	37.70991	127.04391
평균값	37.709885	127.044005



그림 36. 세계지도 모드와 한반도 모드 출력 비교

2. 실험

이번 절에서는 해당 시스템의 정확도를 실험한 결과에 대하여 논할 것이다. 해당 시스템의 정확도를 실험하기 위해서는 빈센티의 공식을 높은 정확도로 계산하는 신뢰도 높은 소프트웨어의 결과와 서로 다른 2개 이상의 좌표가 필요하다. 전자는 Geoscience Australia에서 제공하는 소프트웨어 중 Vincenty's Inverse^[6]의 결과를 참고할 것이다. 그리고 좌표에 대한 실험은 국내에서의 실험과 해외에서의 실험 2가지로 나눌 것이다. 국내에서는 앞에서 구한 기준좌표(신한대학교)와 기준좌표로부터 최소 50km 이상 떨어진 시청 3개, 해외에서는 전세계에서 유명한 6개의 대학교의 좌표를 선택했다. 따라서 국내에서 비교할 때는 각각 기준좌표에서 춘천시청까지, 기준좌표에서 대전광역시청까지, 기준좌표에서 대구광역시청까지의 결과에 대하여 비교할 것이다. 국내에서의 좌표는 표 7과 같다. 해외에서 비교할 때는 각각 동북아시아에서 유럽까지, 아메리카 대륙에서 유럽까지, 유럽 내부에서의 실험 결과를 얻기 위하여 다음과 같이 대학교를 선택하였다. 각각 도쿄 대학교

[6] <https://geodesyapps.ga.gov.au/vincenty-inverse>

에서 괴팅겐 대학교까지, 하버드 대학교에서 케임브리지 대학교까지, 모스크바 국립대학교에서 소르본 대학교까지의 결과를 비교할 것이다. 해외에서의 좌표는 표 8과 같다. 특히 국내에서의 두 좌표 사이의 거리는 지구의 평균 반지름인 6371km 보다 충분히 짧기 때문에 두 좌표 사이의 호는 직선으로 근사할 수 있다. 따라서 시스템에서 계산한 방위각이 실제 지도 위에서 어떤 의미를 갖는지 확인하기 위해 구글지도와 온라인 각도기를 이용하여 확인해볼 것이다.

표 7. 기준좌표 및 국내 주요 시청 좌표

지역명	위도	경도
기준좌표(신한대학교)	37.709885	127.044005
춘천시청	37.8813	127.7303
대전광역시청	36.35	127.3849
대구광역시청	35.8715	128.6015

표 8. 해외 주요 대학 좌표

지역명	위도	경도
도쿄 대학교	35.71377	139.76276
괴팅겐 대학교	51.54147	9.93752
하버드 대학교	42.37444	-71.11823
케임브리지 대학교	52.20539	0.11312
모스크바 국립대학교	55.70418	37.52882
소르본 대학교	48.85061	2.34038

가. 기준좌표에서 춘천시청까지의 실험 결과

그림 37에 따르면 기준좌표에서 춘천시청까지의 거리와 방위각은 각각 63.24km, 72.1°이다.

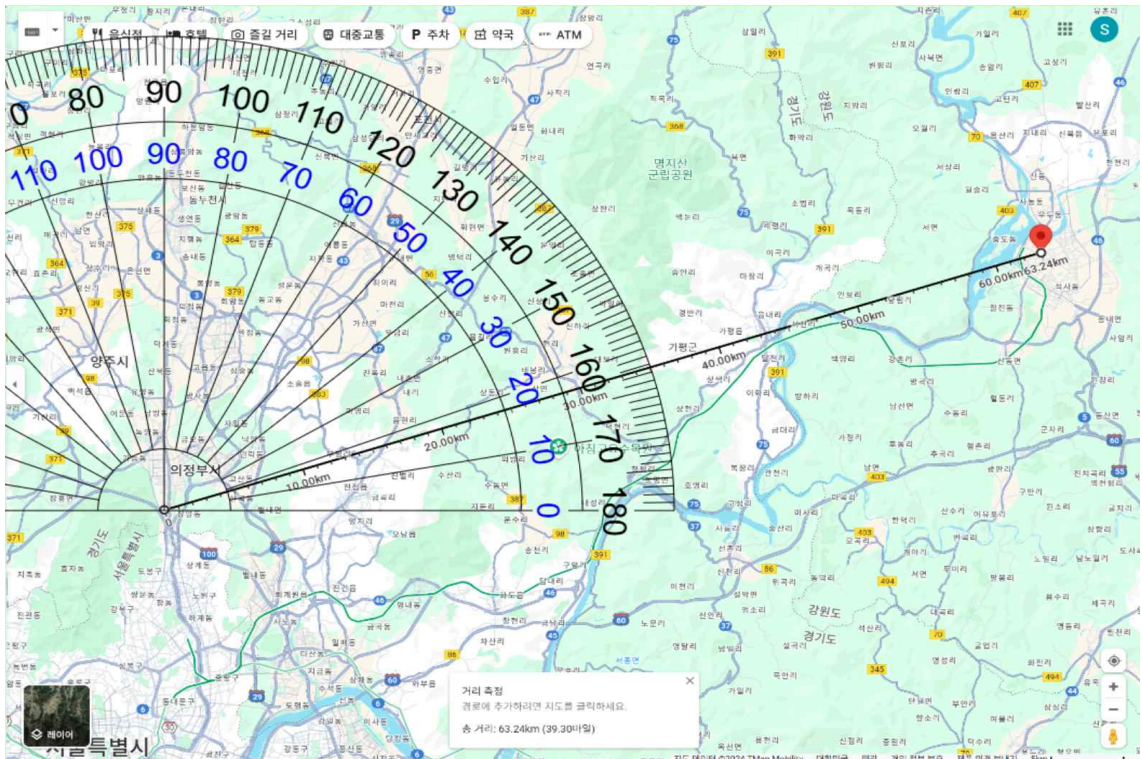


그림 37. 기준좌표에서 춘천시청까지의 구글지도 결과

그림 38, 그림 39, 그림 40은 각각 기준좌표에서 춘천시청까지의 해당 시스템의 하버사인 모드 결과, 빈센티 모드 결과 그리고 Vincenty's Inverse 결과이며 구글 지도의 결과와 비슷한 결과가 출력되었다.



그림 38. 기준좌표에서 춘천시청까지의 하버사인 모드 결과



그림 39. 기준좌표에서 춘천시청까지의 빈센티 모드 결과

Points

Point	Name	Latitude	Longitude
1	Shinhan University	37.709885° N	127.044005° E
2	Chuncheon City Hall	37.8813° N	127.7303° E

Results

Forward Azimuth	Reverse Azimuth	Ellipsoidal Distance
72.318122°	252.738719°	63369.274 metres

그림 40. 기준좌표에서 춘천시청까지의 Vincenty's Inverse 결과

나. 기준좌표에서 대전광역시청까지의 실험 결과

그림 41에 따르면 기준좌표에서 대전광역시청까지의 거리와 방위각은 각각 154.21km, 168.9°이다.

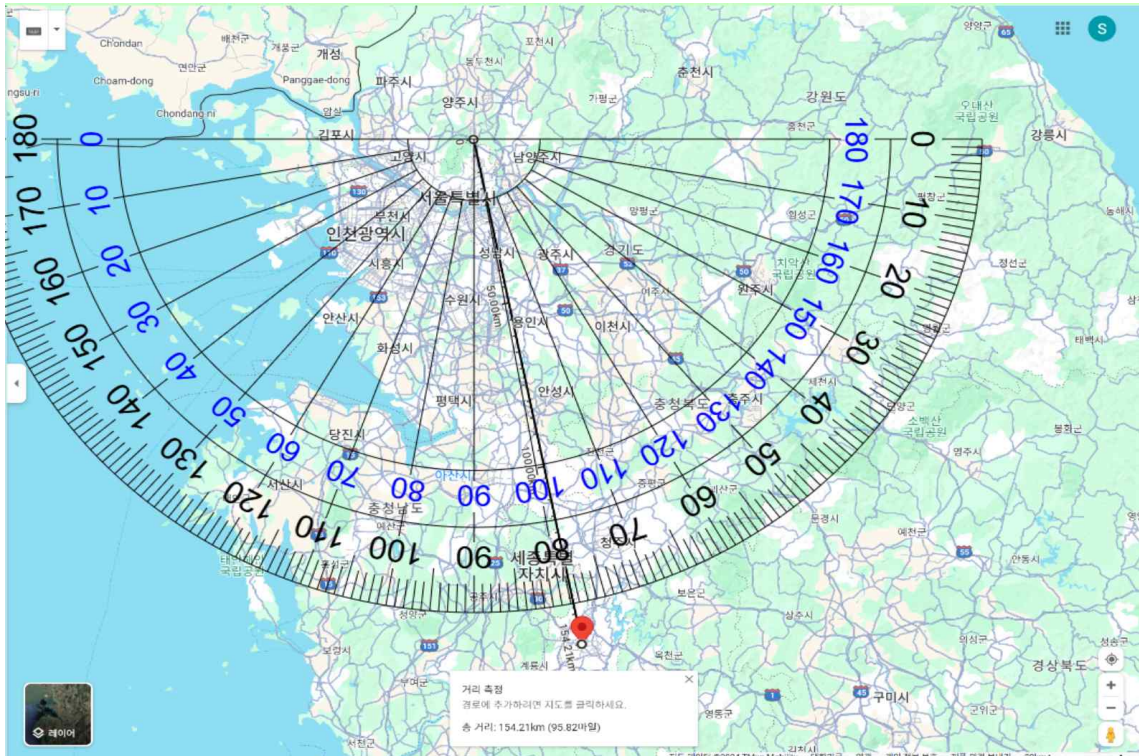


그림 41. 기준좌표에서 대전광역시청까지의 구글지도 결과

그림 42, 그림 43, 그림 44는 각각 기준좌표에서 대전광역시청까지의 해당 시스템의 하버사인 모드 결과, 빈센티 모드 결과 그리고 Vincenty's Inverse 결과이며 구글지도의 결과와 비슷한 결과가 출력되었다.



그림 42. 기준좌표에서 대전광역시청까지의 하버사인 모드 결과



그림 43. 기준좌표에서 대전광역시청까지의 빈센티 모드 결과

Points

Point	Name	Latitude	Longitude
1	Shinhan University	37.709885° N	127.044005° E
2	Daejeon City Hall	36.35° N	127.3849° E

Results

Forward Azimuth	Reverse Azimuth	Ellipsoidal Distance
168.533032°	348.738345°	153935.123 metres

그림 44. 기준좌표에서 대전광역시청까지의 Vincenty's Inverse 결과

다. 기준좌표에서 대구광역시청까지의 실험 결과

그림 45에 따르면 기준좌표에서 대구광역시청까지의 거리와 방위각은 각각 247.02km, 146° 이다.

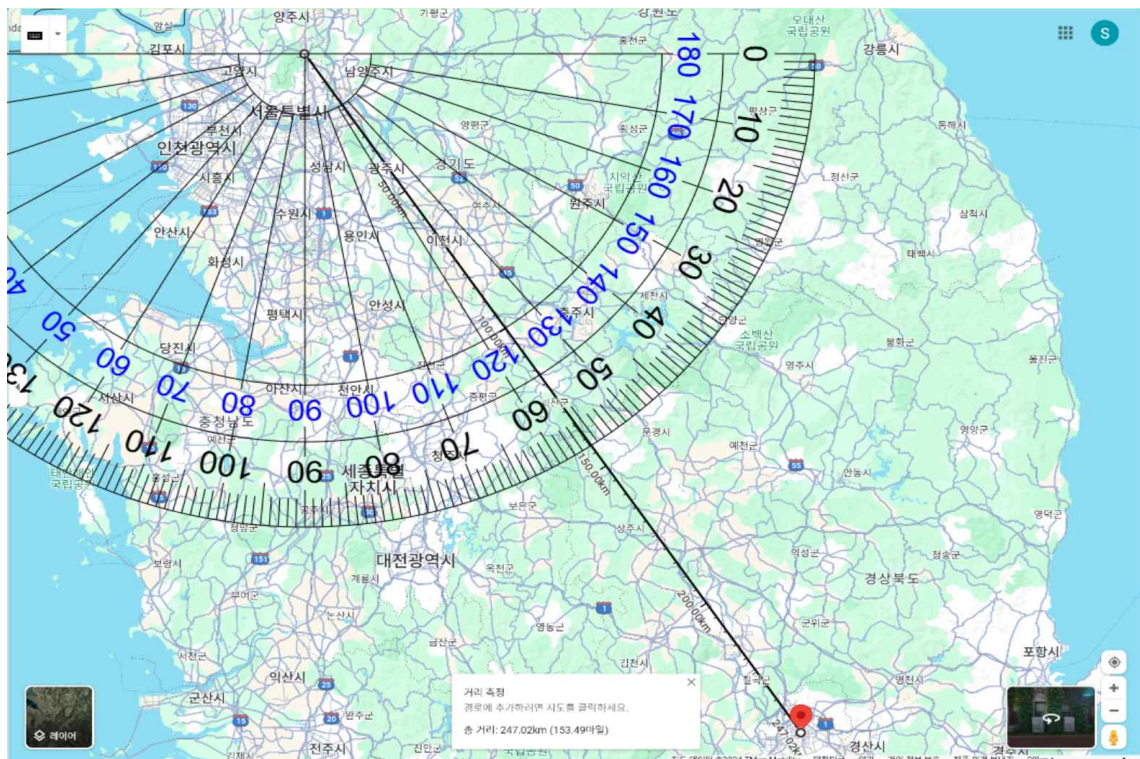


그림 45. 기준좌표에서 대구광역시청까지의 구글지도 결과

그림 46, 그림 47, 그림 48은 각각 기준좌표에서 대구광역시청까지의 해당 시스템의 하버사인 모드 결과, 빈센티 모드 결과 그리고 Vincenty's Inverse 결과이며 구글지도의 결과와 비슷한 결과가 출력되었다.



그림 46. 기준좌표에서 대구광역시청까지의 하버사인 모드 결과



그림 47. 기준좌표에서 대구광역시청까지의 빈센티 모드 결과

Points

Point	Name	Latitude	Longitude
1	Shinhan University	37.709885° N	127.044005° E
2	Daegu City Hall	35.8715° N	128.8015° E

Results

Forward Azimuth	Reverse Azimuth	Ellipsoidal Distance
145.260217°	326.193148°	246862.712 metres

그림 48. 기준좌표에서 대구광역시청까지의 Vincenty's Inverse 결과

라. 도쿄 대학교에서 괴팅겐 대학교까지의 실험 결과

그림 49, 그림 50, 그림 51은 각각 도쿄 대학교에서 괴팅겐 대학교까지의 해당 시스템의 하버사인 모드 결과, 빈센티 모드 결과 그리고 Vincenty's Inverse 결과이다. 해당 시스템에서 Vincenty's Inverse의 결과와 비슷한 값이 출력되었다.



그림 49. 도쿄 대학교에서 괴팅겐 대학교까지의 하버사인 모드 결과



그림 50. 도쿄 대학교에서 괴팅겐 대학교까지의 빈센티 모드 결과

Points

Point	Name	Latitude	Longitude
1	The University of Tokyo	35.71377° N	139.76276° E
2	University of Göttingen	51.54147° N	9.93752° E

Results

Forward Azimuth	Reverse Azimuth	Ellipsoidal Distance
331.184227°	38.951424°	9176010.857 metres

그림 51. 도쿄 대학교에서 괴팅겐 대학교까지의 Vincenty's Inverse 결과

마. 하버드 대학교에서 케임브리지 대학교까지의 실험 결과

그림 52, 그림 53, 그림 54는 각각 하버드 대학교에서 케임브리지 대학교까지의 해당 시스템의 하버사인 모드 결과, 빈센티 모드 결과 그리고 Vincenty's Inverse 결과이다. 해당 시스템에서 Vincenty's Inverse의 결과와 비슷한 값이 출력되었다.



그림 52. 하버드 대학교에서 케임브리지 대학교까지의 하버사인 모드 결과



그림 53. 하버드 대학교에서 케임브리지 대학교까지의 빈센티 모드 결과

Points

Point	Name	Latitude	Longitude
1	Harvard University	42.37444° N	71.11823° W
2	University of Cambridge	52.20539° N	0.11312° E

Results

Forward Azimuth	Reverse Azimuth	Ellipsoidal Distance
52.178067°	287.882743°	5273391.582 metres

그림 54. 하버드 대학교에서 케임브리지 대학교까지의 Vincenty's Inverse 결과

바. 모스크바 국립대학교에서 소르본 대학교까지의 실험 결과

그림 55, 그림 56, 그림 57은 각각 모스크바 국립대학교에서 소르본 대학교까지의 해당 시스템의 하버사인 모드 결과, 빈센티 모드 결과 그리고 Vincenty's Inverse 결과이다. 해당 시스템에서 Vincenty's Inverse의 결과와 비슷한 값이 출력되었다.



그림 55. 모스크바 국립대학교에서 소르본 대학교까지의 하버사인 모드 결과



그림 56. 모스크바 국립대학교에서 소르본 대학교까지의 빈센티 모드 결과

Points

Point	Name	Latitude	Longitude
1	Shinhan University	37.70970° N	127.04605° E
2	Daegu City Hall	35.8715° N	128.6015° E

Results

Forward Azimuth	Reverse Azimuth	Ellipsoidal Distance
145.293144°	326.224848°	246743.121 metres

그림 57. 모스크바 국립대학교에서 소르본 대학교까지의 Vincenty's Inverse 결과

3. 실험결과 정리 및 오차율 계산

이번 절에서는 위에서 살펴본 6가지의 실험 결과를 정리하여 Vincenty's Inverse의 결과에 대한 오차와 오차율을 구하여 시스템이 하버사인 모드와 빈센티 모드에서 각각 얼마나 높은 정확도를 가지는지 확인해볼 것이다. 오차율을 구할 때는 다음의 식을 사용할 것이다. 따라서 모든 오차는 절댓값으로 구할 것이다.

$$\text{Error}(\%) = \frac{|\text{System result} - \text{Software result}|}{\text{Software result}} \times 100\%$$

표 9에서는 수행한 모든 실험에 대한 거리 결과를 정리하였고, 표 10에서는 Vincenty's Inverse를 기준으로 거리에 대한 오차와 오차율을 계산하였다.

표 9. 거리 비교

	하버사인 모드(A)	빈센티 모드(B)	Geodetic Calculators-Vincenty's Inverse(C)
기준좌표->춘천시청	63.22985km	63.18262km	63.369274km
기준좌표->대전광역시청	154.19827km	153.66199km	153.935123km
기준좌표->대구광역시청	247.00066km	246.35876km	246.862712km
도쿄 대학교->괴팅겐 대학교	9153.626km	9159.05370km	9176.010857km
하버드 대학교->케임브리지 대학교	5258.9438km	5261.8892km	5273.391582km
모스크바 국립대학교->소르본 대학교	2481.58570km	2483.56490km	2488.890032km

표 10. 거리 오차 및 오차율

	$ C-A $	$\frac{ C-A }{C} \times 100\%$	$ C-B $	$\frac{ C-B }{C} \times 100\%$
기준좌표->춘천시청	0.139424km	0.2200183%	0.186654km	0.29454969%
기준좌표->대전광역시청	0.263147km	0.17094669%	0.273133km	0.17743384%
기준좌표->대구광역시청	0.137948km	0.05588045%	0.503952km	0.20414262%
도쿄 대학교->괴팅겐 대학교	22.384857km	0.24394977%	16.957157km	0.18479879%
하버드 대학교->케임브리지 대학교	14.447782km	0.27397514%	11.502382km	0.21812114%
모스크바 국립대학교->소르본 대학교	7.304332km	0.29347749%	5.325132km	0.2139561%

표 11에서는 수행한 모든 실험에 대한 방위각 결과를 정리하였고, 표 12에서는 Vincenty's Inverse를 기준으로 방위각에 대한 오차와 오차율을 계산하였다.

표 11. 방위각 비교

	하버사인 모드(D)	빈센티 모드(E)	Geodetic Calculators-Vincenty's Inverse(F)
기준좌표->춘천시청	72.23	72.26	72.318122
기준좌표->대전광역시청	168.59	168.57	168.533032
기준좌표->대구광역시청	145.38	145.32	145.260217
도쿄 대학교->괴팅겐 대학교	331.18	331.13	331.184227
하버드 대학교->케임브리지 대학교	52.15	52.2	52.178067
모스크바 국립대학교->소르본 대학교	266.98	266.99	267.02323

표 12. 방위각 오차 및 오차율

	$ F-D $	$\frac{ F-D }{F} \times 100\%$	$ F-E $	$\frac{ F-E }{F} \times 100\%$
기준좌표->춘천시청	0.088122	0.12185327%	0.058122	0.0803699%
기준좌표->대전광역시청	0.056968	0.03380228%	0.036968	0.0254495%
기준좌표->대구광역시청	0.119783	0.08246098%	0.059783	0.0411558%
도쿄 대학교->괴팅겐 대학교	0.004227	0.00127633%	0.054227	0.01637367%
하버드 대학교->케임브리지 대학교	0.028067	0.0537908%	0.021933	0.0420349%
모스크바 국립대학교->소르본 대학교	0.04323	0.161896%	0.03323	0.01244461%

앞에서 살펴본 수치에 대하여 실험 거리별, 해당 시스템의 모드별로 구분하여 오차율의 평균값을 소수점 둘째 자리까지 구한 결과는 각각 표 13, 14, 15, 16과 같다.

방위각에 대한 오차율은 실험 거리에 상관없이 항상 0.1% 미만의 오차율을 갖고 있다. 또한 하버사인 모드에서 빈센티 모드로 변경시 오차율이 감소한다. 하지만 실험 거리에 상관없이 거리 오차율이 항상 0.2% 부근에 머물러있다. 또한 해외를 기준으로 측정한 장거리 실험에서는 하버사인 모드에서 빈센티 모드로 변경했을 때 오차율이 감소하지만, 국내를 기준으로 측정한 단거리 실험에서는 하버사인 모드에서 빈센티 모드로 변경했을 때 오히려 오차율이 증가한다.

표 13. 국내에서의 하버사인 모드 오차율 평균값

거리 오차율 평균값	0.15%
방위각 오차율 평균값	0.08%

표 14. 국내에서의 빈센티 모드 오차율 평균값

거리 오차율 평균값	0.23%
방위각 오차율 평균값	0.05%

표 15. 해외에서의 하버사인 모드 오차율 평균값

거리 오차율 평균값	0.27%
방위각 오차율 평균값	0.07%

표 16. 해외에서의 빈센티 모드 오차율 평균값

거리 오차율 평균값	0.21%
방위각 오차율 평균값	0.02%

V. 결론

본 논문에서는 자기장센서를 활용한 직선거리 추출시스템을 구현하였다.

구체적으로 이론 결과는 다음과 같다.

1. 하버사인의 공식과 빈센티의 공식을 C언어로 번역하여 시스템에 내장시켜 거리와 방위각을 계산하는데 성공하였다.
2. HMC5883L을 통해 지구 자기장을 측정하여 진행방향에 대한 방위각을 얻어, 현재 위치부터 목표 지점까지의 직선 경로를 예측하는데 성공하였다.
3. 6개의 표본에 대하여 시스템의 모드나 실험 거리에 상관없이 방위각의 오차율은 항상 0.1% 미만이다.
4. 6개의 표본에 대하여 실험 거리에 상관없이 하버사인 모드에서 빈센티 모드로 전환시 방위각의 오차율이 감소한다.

하지만 다음과 같은 문제점이 발견되었다.

1. GPS 모듈이 시간, 날짜, 좌표까지 모두 수신하는데 걸리는 시간은 전원 연결한 후 최소 10분이다.
2. NEO-6M과 같은 저가의 GPS 모듈은 GPRMC 문장에서 속도와 진행방향과 같이 문장의 구성요소의 일부를 수신하지 못한다.
3. MCU가 CH0_GetGPRMC() 함수를 통해 NEO-6M으로부터 받은 데이터 중 GPRMC 문장만을 추출하는데 걸리는 소요시간이 길다.
4. 6개의 표본에 대하여 실험 거리에 상관없이 방위각 오차율은 0.1% 미만인데 반해 거리 오차율은 평균적으로 0.2%이다.
5. 6개의 표본에 대하여 국내에서 측정한 단거리 실험 결과에서 하버사인 모드보다 빈센티 모드에서 오히려 오차율이 증가하였다.

결론적으로 단거리 실험에서는 하버사인 모드가 더 유리하며, 장거리 실험에서는 빈센티 모드가 더 유리하다는 결과가 나왔지만, 표본의 수가 적다는 점에서 쉽게 일반화할 수는 없다. 현재로서는 차량용 내비게이션에 못 미치지만 한반도 내에서 0.2%의 오차율은 짧은 거리이기 때문에 하버사인 모드를 유지하며 간이 내비게이션으로 동작 가능하다. 하지만 위의 문제점들을 해결할 수 있다면 용도에 따라 정확성과 속도 간의 균형, 리소스 절약이 동시에 이루어질 수 있으며 효율적이고 정밀도가 높은 범용적인 거리 및 방위각 계산기로서 지리학 연구 및 군사용 포물선 투사체 발사시스템 등 광범위한 위치 기반 시스템에 사용될 수 있을 것이다.

참고문헌

- [1] Chris Veness, "Calculate distance, bearing and more between Latitude/Longitude points", <https://www.movable-type.co.uk/scripts/latlong.html>
- [2] neha-akhade, "Haversine Vs Vincenty: Which is the best?", <https://www.neovasolutions.com/2019/10/04/haversine-vs-vincenty-which-is-the-best/>
- [3] Wikipedia, "Haversine formula", https://en.wikipedia.org/wiki/Haversine_formula
- [4] Wikipedia, "World Geodetic System", https://en.wikipedia.org/wiki/World_Geodetic_System
- [5] Wikipedia, "Vincenty's formulae", https://en.wikipedia.org/wiki/Vincenty%27s_formulae
- [6] NTREX, "NT-804AN-BW"
- [7] Atmel Corporation, "ATmega128(L) - Complete Datasheet"
- [8] Honeywell, "3-Axis Digital Compass IC HMC5883L"
- [9] u-blox, "NEO-6 u-blox 6 GPS Modules Data Sheet"
- [10] u-blox, "u-blox 6 Receiver Description Including Protocol Specification"
- [11] Solomon Systech, "SSD1309 (Advanced Information, 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller)"